

TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN COMPUTACIÓN



ClearMerge: Resolución de conflictos en proyectos colaborativos de Unity3D

Estudiante: Fernando Álvarez Rodríguez de Legísima

Dirección: Enrique Fernández Blanco

A Coruña, septiembre de 2025.

A todos aquellos que empezaron conmigo y siguen ahí

Agradecimientos

Quiero dar las gracias a esas personas que doy por hecho que están en mi vida aunque puede que algún día ya no sea así. A mi familia por estar siempre para mí aunque a veces no me entiendan, en especial a mis abuelos por su amor incondicional y apoyo siempre impulsándome a ser mejor. Gracias a esos amigos que conozco de poco y los siento de siempre, y aquel que está desde siempre y lo veo muy poco, así como los que me han acompañado toda la carrera.

También quiero agradecer a mi tutor, por guiarme en esta experiencia, aun cuando yo haya tenido algún desacierto, siempre mostrando paciencia y disponibilidad.

Finalmente, gracias a mí yo de hace unos años, que no siempre lo tuvo fácil pero nunca se cansó de intentarlo. Espero estar a la altura de lo que esperabas para este momento, te prometo que me seguiré esforzando.

Resumen

En el desarrollo colaborativo de videojuegos utilizando Unity3D, uno de los problemas más frecuentes es la gestión de **conflictos** al fusionar proyectos modificados por distintos desarrolladores. Unity3D utiliza archivos **YAML** para guardar sus escenas, lo que dificulta las fusiones manuales cuando varios desarrolladores modifican el mismo proyecto simultáneamente. Este trabajo presenta ClearMerge, una herramienta integrada en el editor de Unity3D que simplifica y optimiza el proceso de resolución de conflictos en archivos del proyecto.

ClearMerge proporciona una interfaz visual intuitiva que permite a los desarrolladores detectar automáticamente los conflictos generados por **Git**, visualizar y comparar las versiones conflictivas, y resolver los **conflictos** de forma segura preservando la integridad del proyecto. La herramienta incluye funcionalidades de previsualización en tiempo real, mapeo automático de objetos entre versiones, y un sistema modular que facilita el mantenimiento y la extensibilidad.

La herramienta representa una alternativa gratuita y visual a las soluciones comerciales existentes, proporcionando beneficios tangibles en productividad y reducción de errores.

Abstract

In collaborative video game development using Unity3D, one of the most frequent problems is conflict management when merging projects modified by different developers. Unity3D uses **YAML** files to save its scenes, which makes manual merging difficult when several developers modify the same project simultaneously. This work presents ClearMerge, a tool integrated into the Unity3D editor that simplifies and optimizes the conflict resolution process in project files.

ClearMerge provides an intuitive visual interface that allows developers to automatically detect conflicts generated by **Git**, visualize and compare conflicting versions, and resolve conflicts safely while preserving project integrity. The tool includes real-time preview functionality, automatic object mapping between versions, and a modular system that facilitates maintenance and extensibility.

The tool represents a free and visual alternative to existing commercial solutions, providing tangible benefits in productivity and error reduction.

Palabras clave:

- Unity3D
- Resolución de conflictos
- Control de versiones
- Desarrollo colaborativo
- Interfaz visual
- YAML

Keywords:

- Unity3D
- Conflict resolution
- Version control
- Collaborative development
- Visual interface
- YAML

Índice general

1	Introducción	1
1.1	Motivación	2
1.2	Objetivos	3
1.3	Estructura de la memoria	3
2	Fundamentos	4
2.1	Fundamentos Teóricos	4
2.1.1	Motores de Videojuegos y Desarrollo Colaborativo	4
2.1.2	Control de Versiones	5
2.1.3	Problemática Específica de los conflictos en Videojuegos	6
2.2	Fundamentos Tecnológicos	7
2.2.1	Ecosistema de unity	7
2.2.2	Arquitectura de Archivos en unity	8
2.2.3	Complejidad de los conflictos en Unity	10
2.2.4	Herramientas de Desarrollo	11
3	Estudio de Mercado	12
3.1	Estado del Arte	12
3.1.1	Herramientas Genéricas de Comparación y Fusión	12
3.1.2	Soluciones oficales de Unity	13
3.1.3	Soluciones de Terceros Especializadas	15
3.2	Análisis SWOT	17
4	Metodología y Planificación	19
4.1	Metodología	19
4.2	Adaptación de la Metodología al Proyecto	19
4.3	Planificación de Sprints	20
4.3.1	Sprint 1: Planificación y Análisis	20

4.3.2	Sprint 2: Diseño de Arquitectura	20
4.3.3	Sprint 3: Analizador de Conflictos	21
4.3.4	Sprint 4: Visualizador de Conflictos	21
4.3.5	Sprint 5: Sistema de Resolución	21
4.3.6	Sprint 6: Extensión a Assets	21
4.3.7	Sprint 7: Integración y Documentación	22
4.4	Recursos	22
4.4.1	Recursos Humanos	22
4.4.2	Recursos Técnicos	22
4.4.3	Costes del Proyecto	23
4.5	Planificación Inicial	24
4.6	Resultado del Seguimiento	25
5	Desarrollo	28
5.1	Sprint 1: Planificación y Análisis	28
5.1.1	Análisis	28
5.1.2	Diseño	31
5.1.3	Pruebas	32
5.2	Sprint 2: Diseño de Arquitectura	34
5.2.1	Análisis	34
5.2.2	Diseño	34
5.2.3	Implementación	36
5.2.4	Pruebas	37
5.3	Sprint 3: Analizador de conflictos	37
5.3.1	Análisis	37
5.3.2	Diseño	38
5.3.3	Implementación	40
5.3.4	Pruebas	42
5.4	Sprint 4: Visualizador de Conflictos	44
5.4.1	Análisis	44
5.4.2	Diseño	45
5.4.3	Implementación	50
5.4.4	Pruebas	52
5.5	Sprint 5: Sistema de Resolución	52
5.5.1	Análisis	52
5.5.2	Diseño	53
5.5.3	Implementación	58
5.5.4	Pruebas	59

5.6	Sprint 6: Extensión a Assets	60
5.6.1	Análisis	60
5.6.2	Diseño	61
5.6.3	Implementación	66
5.6.4	Pruebas	68
5.7	Sprint 7: Integración y Documentación	69
5.7.1	Análisis	69
5.7.2	Diseño	70
5.7.3	Implementación	75
5.7.4	Pruebas	76
6	Conclusiones	77
6.1	Objetivos cumplidos	77
6.2	Aprendizajes y aportes personales	78
6.3	Proyección futura	78
7	Futuros Desarrollos	79
7.1	Optimización y Mejoras Técnicas	79
7.2	Otras Mejoras	79
A	Material Adicional	82
A.1	Requisitos funcionales de la herramienta	82
A.2	Casos de uso de la herramienta	83
B	Instalación y Configuración	88
B.1	Requisitos del Sistema	88
B.2	Proceso de Instalación	88
B.3	Configuraciones de Git Recomendadas	89
B.3.1	Archivo .gitignore	89
	Lista de acrónimos	90
	Glosario	91
	Bibliografía	94

Índice de figuras

2.1	Interfaz del motor de videojuego Unity	7
3.1	Comparación de interfaces: Beyond Compare (izquierda) y KDiff3 (derecha) . .	13
3.2	Interfaz de Unity Version Control	14
3.3	Interfaz de GitMerge for Unity mostrando la resolución de conflictos en escenas	15
4.1	Diagrama de Gantt general del proyecto mostrando la distribución de los sprints en el tiempo.	24
4.2	Diagrama de Gantt detallado mostrando tareas específicas, dependencias y hitos críticos.	25
4.3	Diagrama de Gantt: Planificación Final	26
5.1	Actores identificados	32
5.2	Modificaciones paralelas de un mismo GameObject por dos desarrolladores . .	32
5.3	Escena con conflicto tras la fusión de cambios	33
5.4	Diagrama de la estructura general de clases	35
5.5	Caso de uso CU-01: Detectar conflictos en escenas	38
5.6	Analizador y estructuras de datos utilizadas para representar conflictos	39
5.7	Flujo de análisis del ConflictAnalyzer	43
5.8	Casos de uso: Visualización y navegación de conflictos	44
5.9	Diagrama de clases del sistema de visualización de conflictos	46
5.10	Acceso a la herramienta desde el menú Tools > ClearMerge	47
5.11	Ventana Conflict Viewer tras escaneo inicial de conflictos	47
5.12	Ventana Conflict Viewer con escenas cargadas y controles activos	48
5.13	Jerarquía tras cargar ambas escenas de forma aditiva (BEFORE y AFTER) . .	49
5.14	Visualización de ambas escenas cargadas: AFTER (1) y BEFORE (2)	49
5.15	Vista en el Inspector del gameobject conflictivo seleccionado	50
5.16	Casos de uso principales del Sprint 5	53

5.17	Vista de la interfaz durante la previsualización final	54
5.18	Diagrama general del sistema de resolución de conflictos	56
5.19	Configuración de la herramienta en Project Settings	57
5.20	Diagrama de clases del sistema de configuración	58
5.21	Casos de uso del sistema de resolución de conflictos en assets	60
5.22	Interfaz de usuario para resolución de conflictos en assets	64
5.23	Diagrama de clases del sistema de resolución de conflictos en assets	65
5.24	Casos de uso unificados para el Sprint 7	70
5.25	Ventana emergente de alerta de conflictos al iniciar o refrescar el proyecto	71
5.26	Interfaz principal unificada con accesos directos	71
5.27	Asistente de configuración automática al instalar la herramienta	72
5.28	Diagrama de clases de los componentes de configuración y detección automática	73
5.29	Diagrama de clases mostrando la incorporación de <code>CrossReferenceConflict</code> en el módulo de escenas	74

Índice de tablas

2.1	Comparativa entre sistemas de control de versiones utilizados en desarrollo de videojuegos	6
3.1	Comparativa entre herramientas de gestión de versiones para Unity	16
3.2	Matriz Strengths, Weaknesses, Opportunities, and Threats (SWOT) del proyecto	17
4.1	Resumen de roles y responsabilidades asumidos durante el desarrollo	22
4.2	Desglose de costes del proyecto.	23
4.3	Desviación temporal y económica respecto a la planificación inicial.	27
5.1	Requisitos no funcionales del sistema.	29
5.2	Historias de usuario del sistema	29
7.1	Hoja de ruta de optimizaciones técnicas	79
A.1	Requisitos funcionales de la herramienta	82
A.2	Casos de uso identificados para la herramienta	84
A.3	Condiciones y flujo de los casos de uso	85

Introducción

EL desarrollo de videojuegos es un sector laborioso y complejo, ya que implica integrar distintas disciplinas como la animación, el diseño de mundos, la programación, la teoría de juegos, las simulaciones físicas y el diseño de interfaces [UX/UI](#), entre otras, en un único proyecto coordinado. Por esta razón, se considera una actividad multidisciplinar que generalmente requiere la colaboración de grandes equipos formados por profesionales de diversos ámbitos.

Para facilitar esta integración surgieron los denominados motores de videojuegos, herramientas que permiten unificar todos los aspectos del desarrollo, desde el arte hasta la lógica del juego, en un entorno común, con el objetivo de optimizar el trabajo durante el desarrollo. En la industria actual existen numerosos motores, desde aquellos privativos desarrollados desde cero por grandes empresas (como *Frostbite* [1] o *RAGE* [2]), hasta opciones comerciales de pago (como *Unreal Engine* [3] o *CryEngine* [4]), así como alternativas gratuitas y de código abierto (como *Godot* [5]) o limitadas mediante suscripción (como *GameMaker Studio* [6], entre otros). Sin embargo, uno de los motores más utilizados es *Unity3D* [7] ([Unity](#) de ahora en adelante), que es gratuito hasta ciertos umbrales y cuenta con una de las comunidades más grandes del desarrollo de videojuegos, lo que facilita enormemente la incorporación tanto de desarrolladores novatos como de pequeños equipos.

Como ocurre en cualquier proyecto de software, la coordinación entre varios desarrolladores requiere del uso de herramientas conocidas como sistemas de control de versiones. Estas herramientas permiten gestionar los cambios realizados en el código y los recursos del proyecto. Sin embargo, se enfrentan a un problema recurrente: la resolución de conflictos. Estos conflictos ocurren cuando dos o más desarrolladores modifican simultáneamente las mismas partes del proyecto de forma distinta. En el contexto del desarrollo de videojuegos, estos casos son especialmente problemáticos.

Debido a la complejidad interna de los motores de videojuegos, la información almacenada por los archivos del proyecto suele ser difícilmente interpretable, tanto para los sistemas de

control de versiones como para los desarrolladores. En el caso concreto de **Unity**, este utiliza archivos en formato **YAML** para guardar datos relacionados con escenas, **Prefabs** y otros elementos fundamentales del proyecto. Este formato, aunque estructurado, presenta una elevada dificultad para ser editado y comprendido manualmente en caso de **conflicto**, especialmente al carecer de contexto visual sobre los cambios aplicados.

Existen herramientas que se integran o complementan los sistemas de control de versiones y que están especializadas en motores de videojuegos, pero muchas de estas soluciones son de pago. Es el caso de la herramienta oficial de **Unity**, Unity Version Control, que requiere de una suscripción para acceder a todas sus funcionalidades. Aquí radica un problema importante: muchas alternativas gratuitas no ofrecen soluciones adecuadas ni eficientes para gestionar conflictos complejos, o bien presentan limitaciones importantes en su funcionalidad y usabilidad.

Esta situación representa un obstáculo considerable para estudios de videojuegos pequeños, conocidos comúnmente como indies, así como para estudiantes o desarrolladores que utilizan **Unity** con fines no lucrativos. A diferencia de las grandes empresas del sector, estos grupos suelen tener recursos económicos y tiempos de desarrollo limitados, por lo que el acceso a herramientas gratuitas y eficientes para resolver conflictos puede representar una mejora sustancial en su productividad y calidad de trabajo.

1.1 Motivación

La necesidad de una herramienta que simplifique la resolución de **conflictos** en proyectos de **Unity** surge de la experiencia práctica de los equipos de desarrollo que trabajan de forma colaborativa. Las herramientas tradicionales de control de versiones, como **Git**, proporcionan mecanismos básicos para detectar y resolver **conflictos**, pero no están optimizadas para trabajar con la estructura específica de los archivos de **Unity**.

Actualmente existen herramientas para resolver **conflictos** en **Unity**, pero la mayoría no ofrecen funcionalidades avanzadas como una visualización directa del proyecto en tiempo real, son complejas de usar o tediosas de instalar. La herramienta oficial de **Unity**, aunque eficaz, requiere una suscripción, lo cual puede limitar su adopción por parte de equipos con recursos limitados.

La herramienta propuesta se destaca precisamente por brindar una solución gratuita y visual, accesible para cualquier equipo, enfocada para todo tipo de entornos especialmente pequeños equipos de desarrollo y ámbitos educativos.

1.2 Objetivos

El objetivo general de este trabajo es desarrollar una herramienta integrada en el **editor** de Unity que simplifique y optimice el proceso de resolución de **conflictos** en archivos del proyecto durante el desarrollo colaborativo. Esta herramienta proporcionará una experiencia intuitiva y gratuita que permitirá reducir tanto el tiempo como los errores asociados a la resolución manual.

Para alcanzar este objetivo general, se establecen los siguientes subobjetivos específicos:

- Diseñar un sistema de análisis automático de archivos con **conflictos** generados por **Git**.
- Crear un mecanismo de mapeo de **GameObjects** y **componentes** entre versiones en **conflicto**.
- Desarrollar una interfaz gráfica para comparar **GameObjects** conflictivos.
- Implementar un visualizador que permita cargar versiones en paralelo y mostrar cambios en tiempo real.

1.3 Estructura de la memoria

El **Capítulo 1** presenta la motivación del proyecto, los objetivos planteados y la organización general de la memoria.

El **Capítulo 2** desarrolla los fundamentos teóricos y tecnológicos necesarios para comprender el proyecto, incluyendo los conceptos clave y las tecnologías utilizadas.

El **Capítulo 3** analiza las herramientas existentes en el mercado, realiza un estudio del estado del arte y un análisis **SWOT** que justifica la necesidad de la solución propuesta.

El **Capítulo 4** describe la metodología de trabajo adoptada y la planificación inicial del proyecto.

El **Capítulo 5** detalla el proceso de desarrollo iterativo, desde las primeras iteraciones hasta la integración final de las funcionalidades.

El **Capítulo 6** presenta las conclusiones obtenidas y evalúa el grado de cumplimiento de los objetivos planteados.

El **Capítulo 7** expone las posibles líneas de trabajo futuro y mejoras potenciales para la herramienta.

Finalmente, se incluyen apéndices con la guía de usuario, así como secciones de acrónimos y glosario.

Fundamentos

EL marco teórico y tecnológico necesario para comprender el desarrollo de esta herramienta se aborda a través de una revisión de conceptos clave sobre [Unity](#), sistemas de control de versiones y los formatos de archivo que gestiona la herramienta.

2.1 Fundamentos Teóricos

Ahora se describirán los principios teóricos que sustentan el desarrollo colaborativo en videojuegos, así como los conceptos asociados al uso de motores y sistemas de control de versiones.

2.1.1 Motores de Videojuegos y Desarrollo Colaborativo

Existen múltiples herramientas hoy en día para el desarrollo de videojuegos, conocidas como **motores de videojuegos**. Estos motores permiten desarrollar juegos de forma más sencilla, sin necesidad de empezar desde cero, proporcionando herramientas integradas para gráficos, física, audio y sistemas de interacción. [8].

Entre los más conocidos se encuentra [Unity](#) [7], que permite el desarrollo tanto individual como colaborativo en paralelo. Es ampliamente utilizado por estudiantes, desarrolladores indie y grandes estudios de videojuegos. [Unity](#) es gratuito para proyectos pequeños y cuenta con una comunidad extensa que facilita el aprendizaje y la resolución de problemas [9].

El desarrollo de videojuegos, como cualquier proyecto de software, es más eficiente cuando se realiza en paralelo. El trabajo colaborativo mejora la productividad y acelera los tiempos de desarrollo mediante la especialización de tareas y distribución eficiente de la carga de trabajo.

2.1.2 Control de Versiones

Uno de los grandes desafíos del trabajo en paralelo es cómo coordinar y mantener la integridad del proyecto entre múltiples desarrolladores. Para abordar este problema, se utiliza el control de versiones, una herramienta fundamental que permite sincronizar los archivos del proyecto, asegurando que todos los miembros del equipo trabajen con la versión más actualizada y fiable [10]. Además, contribuye a conservar la integridad y coherencia del proyecto, facilitando la colaboración eficiente y reduciendo conflictos entre los cambios realizados por distintos desarrolladores [11].

Control de Versiones en el Desarrollo de Videojuegos

El desarrollo de videojuegos, como cualquier proyecto de software, es más eficiente cuando se realiza en paralelo. El trabajo colaborativo mejora la productividad y acelera los tiempos de desarrollo mediante la especialización de tareas y distribución eficiente de la carga de trabajo. En este contexto, existen diversas opciones para implementar un sistema de **Version Control System (Sistema de Control de Versiones) (VCS)**, siendo *Git* [12] el más utilizado tanto dentro como fuera del ámbito profesional, razón por la cual será el enfoque principal de este proyecto. No obstante, en entornos de gran escala, como los de los **Juegos AAA**, también se emplean herramientas como *Perforce* [13] o *SVN* [14], que ofrecen características mejor adaptadas a la gestión de grandes volúmenes de datos y **archivo binario**, frecuentes en este tipo de desarrollos.

Una de las limitaciones de *Git* frente a estos sistemas es su gestión nativa de **archivos binarios**. Para mitigar esta carencia, existen soluciones como **Git Large File Storage (Git LFS)** [15], que permite almacenar archivos pesados en repositorios externos. Sin embargo, la herramienta desarrollada en este proyecto se ha concebido con el objetivo de integrarse de forma sencilla en flujos de trabajo convencionales de *Git* sin depender de configuraciones adicionales o extensiones externas, priorizando así la simplicidad y la accesibilidad para pequeños equipos o desarrolladores individuales.

La tabla 2.1 refleja la comparativa de todas las herramientas mencionadas.

Sistema	Soporte binarios	Distribuido	Facilidad de uso	Licencia
Git	Bajo (sin LFS)	Sí	Alta	Libre (GPL)
Git + LFS	Medio/Alto	Sí	Media	Libre + hosting externo
Perforce	Alto	No	Media/Baja	Propietaria
SVN	Medio	No	Media	Libre

Tabla 2.1: Comparativa entre sistemas de control de versiones utilizados en desarrollo de videojuegos

2.1.3 Problemática Específica de los conflictos en Videojuegos

Aunque existen mecanismos para resolver [conflictos](#) en control de versiones, en el contexto de videojuegos las soluciones tradicionales pueden resultar insuficientes debido a la naturaleza compleja e interdependiente de los recursos involucrados. Esta problemática se manifiesta en tres aspectos fundamentales que dificultan significativamente la gestión de [conflictos](#) [16].

Complejidad estructural de los datos. Los archivos de videojuegos contienen estructuras de datos complejas con relaciones jerárquicas entre objetos. A diferencia de los archivos de código tradicionales, donde las líneas son relativamente independientes, los archivos de [Unity](#) presentan dependencias anidadas donde la modificación de un elemento puede afectar múltiples componentes relacionados, creando conflictos en cascada que no son evidentes a primera vista [17].

Referencias cruzadas entre recursos. Los elementos de un videojuego mantienen referencias cruzadas extensivas entre sí. Cuando se producen conflictos que involucran varios recursos, las herramientas tradicionales como [Git](#) no pueden comprender estas relaciones semánticas, resultando en resoluciones que pueden romper la integridad del proyecto [16].

Diversidad de lógicas de juego. En el desarrollo de videojuegos, cada equipo suele diseñar su propia lógica de juego y arquitectura de proyecto, lo que complica la creación de una herramienta completamente automatizada. Esta diversidad genera lo que se conoce como “re-

laciones invisibles” entre los recursos del juego, donde las dependencias funcionales no están explícitamente declaradas en los archivos, dificultando su detección y gestión automática [18].

Estos tres problemas convergen para crear un escenario donde las herramientas tradicionales de resolución de conflictos resultan inadecuadas, requiriendo soluciones especializadas que comprendan la naturaleza específica de los proyectos de videojuegos.

2.2 Fundamentos Tecnológicos

Se detallan las herramientas y tecnologías utilizadas en el proyecto, con énfasis en la arquitectura de **Unity**, el control de versiones y la estructura interna de los archivos que componen un videojuego.

2.2.1 Ecosistema de unity

El editor de **Unity** está compuesto por un sistema modular de ventanas, el cual se utilizara a lo largo del desarrollo, ya que el proyecto se apoyara en la **Interfaz de Programación de Aplicaciones (API)** que **Unity** ofrece para quienes deseen personalizarlo. En la figura 2.1, se muestran las partes más importantes del editor.

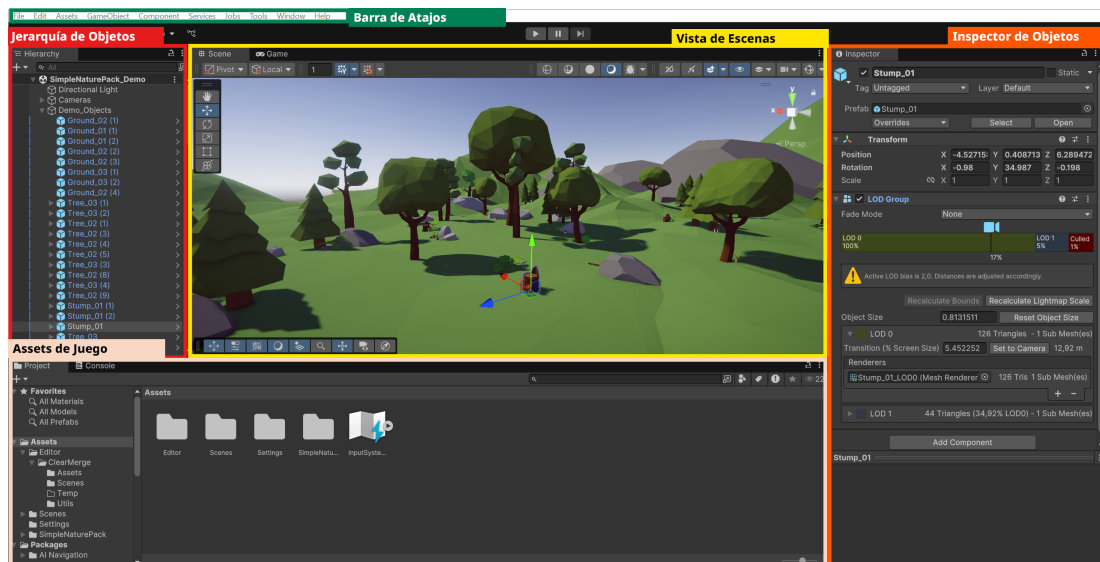


Figura 2.1: Interfaz del motor de videojuego **Unity**

En la Figura 2.1 se pueden identificar varias áreas clave que conforman el entorno de trabajo de **Unity**. Para comprender mejor el desarrollo, se describen brevemente las más relevantes:

- **Assets del juego:** Incluye todos los archivos binarios, escenas y scripts utilizados durante el desarrollo del juego.
- **Vista de escena:** Muestra la(s) escena(s) actualmente cargada(s) en el editor.
- **Jerarquía de objetos:** Presenta todos los **GameObjects** contenidos dentro de la escena activa.
- **Inspector de objetos:** Muestra las propiedades específicas del **GameObject** seleccionado en la escena, así como de cualquier **asset** seleccionado en la ventana de **assets**.
- **Barra de atajos:** Contiene accesos directos a las distintas ventanas de **Unity**; se utilizara para integrar nuestra propia herramienta.

2.2.2 Arquitectura de Archivos en unity

Unity organiza sus proyectos mediante un sistema de archivos estructurado, donde cada archivo representa un **asset** del videojuego, como modelos, texturas, scripts, sonidos, entre otros. Entre estos, destacan los archivos especiales llamados **archivos de escena** (con extensión `.scene`), que representan niveles o mundos del juego, incluyendo la disposición de **GameObjects** y sus propiedades.

Unity utiliza un sistema basado en escenas 2D o 3D, conocidas como *escenas*, que definen un mundo o nivel en el cual existen distintos objetos denominados **GameObjects**. Cada **GameObject** puede contener múltiples **componentes**, ya sean propios del motor o personalizados por el desarrollador, que definen su comportamiento, apariencia o interacción. Estos objetos se organizan jerárquicamente, lo que permite que unos actúen como padres y otros como hijos anidados, facilitando la composición estructural de la escena.

Entre los **assets** más relevantes se encuentran los *prefabs*, que son plantillas reutilizables de **GameObjects** con una configuración específica de **componentes**. Un **Prefab** permite instanciar múltiples copias idénticas de un objeto manteniendo un vínculo con el original, lo que facilita cambios en masa y promueve una mayor consistencia en el diseño del juego. Al modificar el **Prefab** base, todas sus instancias pueden actualizarse automáticamente, salvo que se hayan realizado sobreescrituras locales.

Los archivos de escena funcionan como contenedores que encapsulan la configuración espacial de los **GameObjects**, la iluminación, las cámaras y todas las instancias de **assets** que forman un nivel particular del videojuego. Estos archivos están escritos en formato **YAML**, un formato de texto legible tanto para humanos como para sistemas de control de versiones.

El código mostrado en el Listado 2.1 representa una estructura típica de un archivo de escena en **Unity**:

```

1 %yaml 1.1
2 %TAG !u! tag:unity.com,2011:
3 --- !u!29 &1
4 OcclusionCullingSettings:
5   m_ObjectHideFlags: 0
6   serializedVersion: 2
7   m_OcclusionBakeSettings:
8     smallestOccluder: 5
9     smallestHole: 0.25
10 --- !u!104 &2
11 RenderSettings:
12   m_ObjectHideFlags: 0
13   m_Fog: 0
14   m_FogColor: {r: 0.5, g: 0.5, b: 0.5, a: 1}
15 --- !u!1 &519420028
16 gameobject:
17   m_ObjectHideFlags: 0
18   m_CorrespondingSourceObject: {fileID: 0}
19   m_PrefabInstance: {fileID: 0}
20   m_PrefabAsset: {fileID: 0}
21   serializedVersion: 6
22   m_Component:
23     component: {fileID: 519420032}
24     component: {fileID: 519420031}
25   m_Layer: 0
26   m_Name: Main Camera
27   m_TagString: MainCamera

```

Listing 2.1: Fragmento representativo de un archivo de escena de unity

Por otro lado, los archivos `.meta` permiten que **Unity** mantenga referencias estables a los **assets** incluso si se cambia el nombre o la ubicación del archivo original. Estos archivos contienen los **GUIDs** (identificadores únicos globales) para cada **asset**, configuraciones específicas de importación, metadatos de versión y referencias a dependencias.

El Listado 2.2 muestra un ejemplo típico de archivo `.meta` asociado a un **asset**:

```

1 fileFormatVersion: 2
2 guid: 2cda990e2423bbf4892e6590ba056729
3 TextureImporter:
4   internalIDToNameTable: []
5   externalObjects: {}
6   serializedVersion: 11
7   mipmaps:
8     mipMapMode: 0
9     enableMipMap: 1
10  textureFormat: 1

```



```

11  maxTextureSize: 2048
12  textureSettings:
13    serializedVersion: 2
14    filterMode: 1
15    aniso: 1
16    mipBias: 0
17    wrapU: 0
18    wrapV: 0

```

Listing 2.2: Ejemplo de archivo `.meta` con `guid` y configuraciones de importación

Los **GUIDs** son fundamentales para que **Unity** mantenga la integridad de las referencias entre **assets**, sin importar si sus nombres o ubicaciones cambian. Estos identificadores se generan automáticamente y permanecen inalterables durante toda la vida útil del **asset**, lo que facilita la gestión y la integración con sistemas de control de versiones como **Git**.

El uso del formato **YAML** permite representar estructuras de datos complejas con propiedades anidadas y referencias cruzadas entre **GameObjects**. Sin embargo, esta flexibilidad también implica que el formato es sensible a errores de indentación o espacios, lo que puede causar problemas si se modifica incorrectamente.

2.2.3 Complejidad de los conflictos en Unity

No obstante, los **conflictos en Unity** son más complejos que en otros proyectos de software debido a que los archivos y **assets** están altamente interrelacionados y dependen unos de otros. La sintaxis de **YAML**, aunque legible para humanos, puede ser tediosa y difícil de interpretar cuando se presentan diferencias o conflictos. Las herramientas tradicionales de **Git**, que operan a nivel de línea sin comprender la estructura semántica del archivo, no resultan suficientes para resolver estos conflictos de manera segura y eficiente.

Incluso conflictos aparentemente simples, como una discrepancia en la escala de un **GameObject**, pueden ser difíciles de resolver manualmente debido a la estructura del archivo **YAML**. El Listado 2.3 muestra un ejemplo típico de conflicto generado durante un **merge** en un archivo de escena de **Unity**:

```

1  --- !u!4 &1001
2  Transform:
3    m_ObjectHideFlags: 0
4    m_LocalPosition: {x: 0, y: 1, z: 0}
5  <<<<<<< HEAD
6    m_LocalScale: {x: 1, y: 1, z: 1}
7  =====
8    m_LocalScale: {x: 1.5, y: 1.5, z: 1.5}
9  >>>>>>>
10 m_LocalRotation: {x: 0, y: 0, z: 0, w: 1}

```

```

11  m_Children: []
12  m_Father: {fileID: 0}
13  m_RootOrder: 0
14  m_LocalEulerAnglesHint: {x: 0, y: 0, z: 0}

```

Listing 2.3: Conflicto de versiones en la escala de un GameObject en unity

En este ejemplo, aunque el cambio solo afecta a la escala, la lectura y combinación correcta de estas líneas requiere atención cuidadosa para evitar inconsistencias o daños en la escena. Por ello, la resolución de conflictos en archivos de **Unity** demanda herramientas especializadas o procesos manuales detallados que comprendan la estructura interna de los **assets**.

2.2.4 Herramientas de Desarrollo

Las herramientas técnicas necesarias para el desarrollo de este proyecto incluyen tanto hardware como software especializado, optimizados para el desarrollo en el ecosistema **Unity**. A continuación se detallan las herramientas y tecnologías utilizadas:

- **Unity 6** [19]: Plataforma principal de desarrollo que proporciona el entorno de ejecución y las APIs necesarias para la integración del sistema de resolución de conflictos .
- **Visual Studio 2022 Community** [20]: Entorno de desarrollo integrado optimizado para C#, con capacidades para diseño y generación de diagramas integrados .
- **Git** [12]: Sistema de control de versiones distribuido fundamental para la gestión del código fuente y colaboración en el desarrollo.
- **GitHub** [21]: Plataforma de hospedaje remoto que facilita la gestión de issues, documentación y coordinación con validadores externos .
- **Instagantt** [22]: Herramienta utilizada para la planificación y seguimiento del proyecto mediante diagramas de Gantt .
- **Draw.io** [23]: Aplicación para la creación de diagramas de casos de uso y documentación visual del sistema .
- **Mermaid** [24]: Herramienta para la generación de diagramas de flujo de forma sencilla y automática .
- **Asset Store de Unity** [25]: Se utilizaron **assets** específicos para pruebas, en particular el paquete *Simple Nature Pack* [26], que permitió crear entornos naturales de baja poligonización para el testeo del sistema.

Estudio de Mercado

3.1 Estado del Arte

El panorama actual de herramientas para la resolución de conflictos en proyectos [Unity](#) presenta una diversidad de enfoques que van desde herramientas genéricas adaptadas al contexto específico del desarrollo de videojuegos hasta soluciones oficiales integradas. La comprensión detallada de este ecosistema resulta fundamental para identificar las brechas existentes y las oportunidades de mejora que puede abordar una nueva solución.

3.1.1 Herramientas Genéricas de Comparación y Fusión

Se consideran herramientas genéricas aquellas que no están especializadas en resolver conflictos en desarrollos de videojuegos, sino más bien en ampliar y facilitar la gestión de conflictos en un entorno general.

Herramientas Genéricas de Comparación

Las herramientas genéricas como **Beyond Compare** [27] y **KDiff3** [28] representan soluciones establecidas en el ámbito de la comparación y fusión de archivos. Beyond Compare ofrece una solución comercial madura con capacidades robustas de comparación visual, mientras que KDiff3 proporciona una alternativa de código abierto con funcionalidades fundamentales de fusión de tres vías.

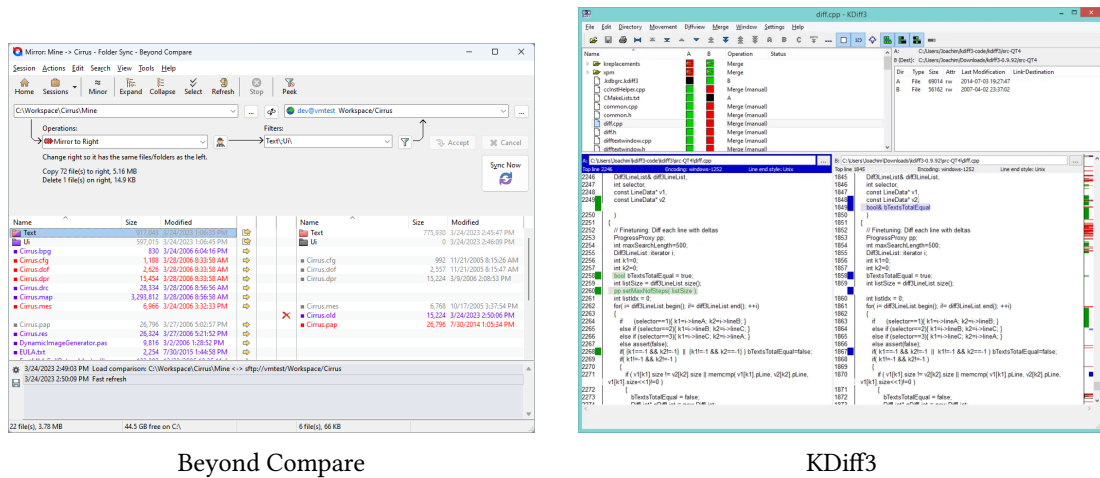


Figura 3.1: Comparación de interfaces: Beyond Compare (izquierda) y KDiff3 (derecha)

Ambas herramientas ofrecen capacidades de comparación de archivos de texto y compatibilidad multiplataforma, pero comparten limitaciones significativas para el contexto específico de **Unity**. Además de ser externas y no estar integradas en el editor, carecen de interpretación semántica de archivos **YAML** de **Unity**, tratándolos como texto genérico, y requieren conocimientos técnicos avanzados para comprender las diferencias en archivos de escena complejos. Beyond Compare añade una barrera económica con su licencia comercial, mientras que KDiff3, aunque gratuito, presenta una interfaz menos intuitiva para usuarios no técnicos.

3.1.2 Soluciones oficiales de Unity

Unity consta de varias opciones oficiales, que mejoran aspectos en mayor o menor medida.

UnityYamlMerge

Unity [7] proporciona UnityYAMLMerge [29], una herramienta oficial diseñada específicamente para la resolución de conflictos en archivos **YAML** generados por el motor **Unity**. Ofrece integración nativa con el editor **Unity** y compatibilidad con múltiples sistemas de control de versiones, incluyendo **Git** [12], **Perforce** [13] y **SVN** [14].

Sin embargo, presenta limitaciones significativas: carece de interfaz visual para conflictos complejos, requiere conocimientos técnicos profundos para trabajar directamente con código **YAML**, y necesita configuración manual para la integración en el flujo de trabajo. Estas limitaciones representan oportunidades claras para el desarrollo de soluciones mejoradas.

Unity Version Control

Unity Version Control [30] es el resultado de la evolución de Plastic SCM [31], un sistema de control de versiones distribuido desarrollado originalmente por la empresa Codice Software [32]. Plastic SCM fue diseñado específicamente para entornos con grandes volúmenes de archivos, especialmente binarios, lo que lo convirtió en una herramienta popular en el desarrollo de videojuegos. Su enfoque visual y su capacidad para gestionar repositorios complejos lo diferenciaban de soluciones más genéricas como Git [12].

En 2020, Unity Technologies adquirió Codice Software [33] con el objetivo de integrar su tecnología dentro del ecosistema Unity. A partir de esta adquisición, Plastic SCM fue rebautizado como Unity Version Control [30] y pasó a formar parte del conjunto de herramientas oficiales ofrecidas por Unity. Esta transición no solo implicó un cambio de nombre, sino también una profunda adaptación del sistema para mejorar su integración con el editor Unity como se ve en la figura 3.2 y facilitar su adopción por parte de equipos multidisciplinares.

Esta herramienta trasciende la simple resolución de conflictos para ofrecer un ecosistema completo de gestión de proyectos colaborativos.

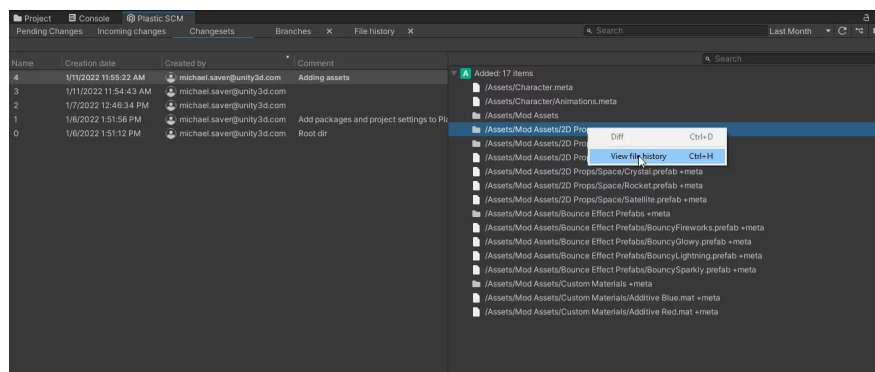


Figura 3.2: Interfaz de Unity Version Control

Las capacidades avanzadas de fusión para escenas y Prefabs incorporan algoritmos especializados que comprenden la semántica específica de los archivos Unity, permitiendo resoluciones automáticas más sofisticadas que las posibles con herramientas genéricas. La visualización de ramas y cambios proporciona una perspectiva histórica clara del desarrollo del proyecto, facilitando la comprensión de la evolución del código y la identificación de puntos de divergencia.

El manejo eficiente de archivos grandes y binarios, común en proyectos de videojuegos que incluyen assets gráficos y de audio de gran tamaño, optimiza el rendimiento del sistema de control de versiones y reduce los tiempos de sincronización. La integración directa con Unity elimina la fricción entre el desarrollo y la gestión de versiones, proporcionando una experiencia de usuario unificada.

No obstante, las limitaciones económicas constituyen el principal obstáculo para la adopción generalizada de esta solución. La necesidad de suscripción para poder usar la herramienta crea una barrera económica significativa, especialmente para estudios independientes y equipos educativos con presupuestos limitados.

3.1.3 Soluciones de Terceros Especializadas

Otras comunidades o empresas externas han implementado soluciones diversas al problema.

GitMerge for Unity

El ecosistema de código abierto ha producido varias iniciativas dirigidas a abordar las limitaciones de las herramientas oficiales, siendo GitMerge for Unity[34] una de las propuestas más prominentes. Esta herramienta representa un enfoque comunitario para resolver los desafíos específicos de la fusión en proyectos [Unity](#), combinando la accesibilidad del código abierto con la especialización técnica requerida.

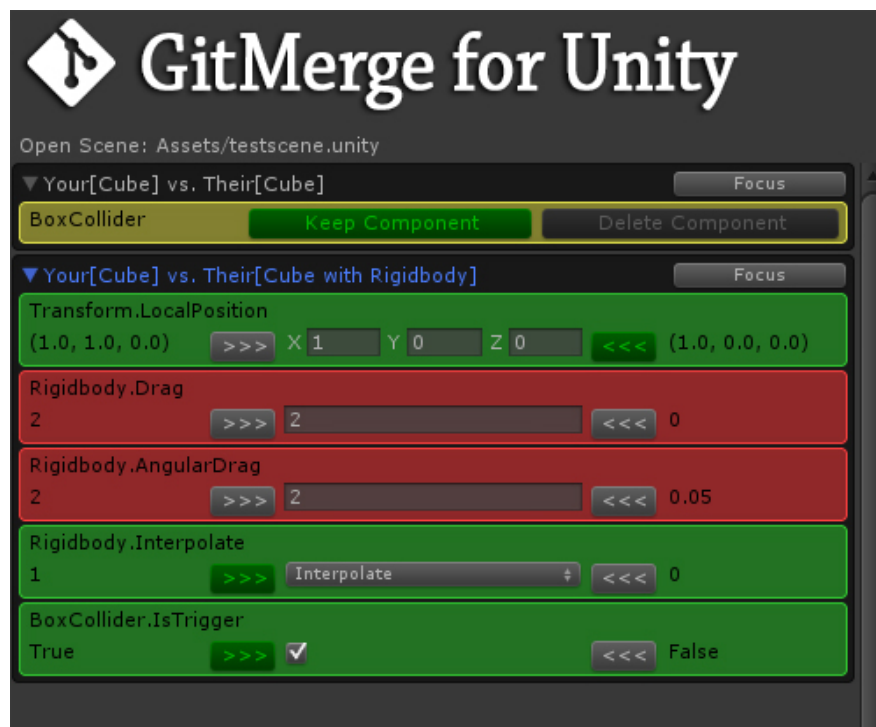


Figura 3.3: Interfaz de GitMerge for Unity mostrando la resolución de conflictos en escenas

La interfaz está especializada y ofrece una resolución mucho más comprensible de los conflictos que pueden surgir en [Prefabs](#) y escenas del proyecto. Presenta las diferencias de-

tectadas en forma de lista clara y estructurada, dentro de una ventana integrada en el propio editor de [Unity](#). De este modo, la herramienta se incorpora de forma natural al flujo de trabajo del desarrollador. En la Figura 3.3 se muestra un ejemplo de esta interfaz, donde es posible visualizar y resolver los conflictos directamente desde el entorno de desarrollo.

Una herramienta que puede considerarse como competidora directa de este proyecto es una solución de código abierto que cuenta con una comunidad activa, aunque pequeña, y presenta algunas limitaciones relevantes. Uno de sus principales problemas es la falta de compatibilidad clara con las distintas versiones de [Unity](#), una cuestión reconocida incluso por los propios desarrolladores en su repositorio:

After five long years, [Unity](#) suddenly fixed issue #17 for us. The project is now back to a state full of potential, but it has years of [Unity](#) updates to catch up to.

Además, carece de algunos de los pilares fundamentales que definen este proyecto, como la simplicidad en el proceso de instalación o la capacidad de visualizar en tiempo real los cambios realizados sobre la escena. Esto se puede observar en la tabla 3.1.

Tabla 3.1: Comparativa entre herramientas de gestión de versiones para Unity

Herramienta	Integrada en Editor	Conflictos visuales	Binarios	Licencia
Beyond Compare	No	No	No	Comercial
KDiff3	No	No	No	Gratuita
UnityYAMLMerge	Sí	No	No	Gratuita
Unity Version Control	Sí	Avanzada	Todo tipo	Suscripción
GitMerge for Unity	Sí	Simplificada	Limitado	Gratuita
Propuesta	Sí	Visual e interactiva	Limitado	Gratuita

3.2 Análisis SWOT

El análisis **SWOT** (Strengths, Weaknesses, Opportunities, Threats) proporciona un marco estructurado para evaluar la posición estratégica de este proyecto en el contexto competitivo actual. Esta evaluación considera tanto factores internos inherentes al diseño y implementación de la herramienta como factores externos del entorno de mercado que pueden influir en su adopción y éxito.

Fortalezas	Debilidades
• Interfaz visual intuitiva que simplifica el proceso de resolución. • Integración directa con el entorno de trabajo de Unity. • Visualización comparativa que facilita la toma de decisiones. • Naturaleza de código abierto y diseño modular. • Herramienta completamente gratuita. • Posibilidad de previsualizar el resultado final en tiempo real.	• Posibles limitaciones al manejar conflictos complejos o numerosos. • Potencial impacto en el rendimiento con proyectos de gran tamaño. • Problemas de compatibilidad con versiones antiguas de Unity. • Compatibilidad restringida a Git como sistema de control de versiones.
Oportunidades	Amenazas
• Demanda creciente de herramientas para trabajo colaborativo en Unity. • Falta de soluciones visuales e intuitivas para conflictos de fusión. • Potencial adopción en entornos académicos, formativos y profesionales. • Reducción significativa del tiempo de trabajo y riesgo de errores en la integración.	• Cambios estructurales en Unity que invaliden parte del funcionamiento. • Aparición de nuevas herramientas oficiales o de terceros con funcionalidades similares. • Complejidad variable de los conflictos generados por diferentes flujos de trabajo con Git.

Tabla 3.2: Matriz **SWOT** del proyecto

Presentada la matriz **SWOT** en la tabla 3.2, podemos concluir que el desarrollo de esta herramienta está justificado, ya que responde a necesidades reales no cubiertas por soluciones actuales. La ausencia de herramientas visuales e intuitivas para resolver conflictos en **Unity**, junto con la creciente demanda de colaboración en equipos, refuerzan su oportunidad. Además, sus fortalezas, como la previsualización en tiempo real y la integración directa con el editor, abordan directamente problemas frecuentes en el flujo de trabajo. Aunque existen algunas limitaciones técnicas, estas no comprometen el valor que aporta la herramienta en escenarios reales de desarrollo.

Metodología y Planificación

EL desarrollo de la herramienta se organiza siguiendo una versión simplificada del marco ágil [Scrum](#) [35], adaptada a un entorno de desarrollo individual. Esta metodología permite trabajar de forma iterativa y flexible, ideal para proyectos con descubrimientos técnicos frecuentes y requisitos en evolución constante, como es el caso de una herramienta experimental para [Unity](#).

4.1 Metodología

En lugar de aplicar [Scrum](#) con todos sus elementos formales, se ha optado por una simplificación centrada en tres pilares clave: el *Product Backlog*, el concepto de *Sprint* como unidad de trabajo, y las sesiones de *Sprint Review*. Esto permite mantener una organización clara y un seguimiento continuo del progreso sin imponer una carga innecesaria al trabajar en solitario.

4.2 Adaptación de la Metodología al Proyecto

El desarrollo se organizará en varios *sprints*, cada uno con una duración limitada y centrado en la entrega de valor incremental. En cada sprint se seguirá un mismo ciclo estructurado en las siguientes etapas:

- **Análisis:** Al inicio de cada sprint se revisarán las prioridades del *product backlog*, detallando las funcionalidades a implementar. Esta etapa se centrará en comprender las necesidades específicas de los usuarios finales y en refinar las historias de usuario, ajustando el alcance de acuerdo con los objetivos del sprint.
- **Diseño:** Una vez definidas las tareas a abordar, se elaborarán los diagramas necesarios ([Unified Modeling Language \(UML\)](#), esquemas de flujo, interfaces) que facilitarán la planificación de la solución técnica. Esta fase permitirá visualizar la arquitectura interna de los módulos implicados y prever la integración con el sistema existente.

- **Implementación:** Se procederá al desarrollo de las funcionalidades acordadas, aplicando buenas prácticas de programación y manteniendo una estructura modular y escalable. El código se construirá de forma incremental, priorizando el cumplimiento de los requisitos definidos en las historias de usuario.
- **Pruebas:** Durante y al finalizar la implementación, se llevarán a cabo pruebas para verificar la funcionalidad y fiabilidad del sistema. Se utilizarán casos de prueba reales y criterios de aceptación previamente definidos. Esta etapa permitirá detectar errores tempranos y garantizar la estabilidad del producto.

Este ciclo se repetirá en cada sprint, lo que permitirá una evolución progresiva del sistema y facilitará el ajuste continuo a medida que surjan nuevas necesidades, descubrimientos técnicos o mejoras de diseño. Además, al final de cada sprint se realizarán sesiones de revisión y retrospectiva para evaluar el trabajo realizado y planificar mejoras tanto en la dinámica del equipo como en el desarrollo del producto.

4.3 Planificación de Sprints

El desarrollo de esta herramienta se estructura en siete sprints con objetivos específicos que evolucionan desde la comprensión del problema hasta la implementación completa del sistema.

4.3.1 Sprint 1: Planificación y Análisis

Duración: 2.5 semanas **Semanas:** 1–3

Este sprint inicial se centra en establecer los fundamentos sólidos del proyecto mediante un análisis exhaustivo del dominio de conflictos en [Unity](#). El objetivo principal es comprender profundamente la estructura de archivos [YAML](#) de [Unity](#), identificar patrones de conflicto típicos en entornos colaborativos y establecer los requisitos funcionales que guiarán el desarrollo posterior. Se desarrolla un conocimiento especializado sobre la semántica interna de [Unity](#), se recopilan casos reales de conflictos para validación y se define la especificación completa de requisitos.

4.3.2 Sprint 2: Diseño de Arquitectura

Duración: 2.5 semanas **Semanas:** 4–6

El segundo sprint establece la arquitectura fundamental del sistema y define los componentes principales. Se diseña la estructura modular que permitirá la evolución del sistema, estableciendo las interfaces y patrones de diseño que guiarán la implementación posterior.

Este sprint transforma el conocimiento adquirido en el sprint anterior en una base arquitectónica sólida para el desarrollo de los componentes funcionales.

4.3.3 Sprint 3: Analizador de Conflictos

Duración: 3 semanas **Semanas:** 7–9

Este sprint desarrolla el núcleo del sistema de análisis de conflictos. Se implementa el parser [YAML](#) especializado capaz de interpretar la estructura compleja de archivos [Unity](#) y el detector automático de marcadores de conflicto [Git](#). El sistema desarrolla capacidades de análisis semántico y detección inteligente de conflictos usando como base los marcadores [Git](#), proporcionando la base para la resolución inteligente de conflictos.

4.3.4 Sprint 4: Visualizador de Conflictos

Duración: 3.5 semanas **Semanas:** 10–13

El cuarto sprint se enfoca en desarrollar la experiencia de visualización que permite a los usuarios comprender los conflictos identificados. Se implementa el sistema de visualización dual que permite comparar simultáneamente ambas versiones conflictivas en el editor [Unity](#), con navegación automática entre [GameObjects](#) conflictivos y herramientas de enfoque que facilitan la comprensión del problema antes de proceder a su resolución.

4.3.5 Sprint 5: Sistema de Resolución

Duración: 2.5 semanas **Semanas:** 14–16

Este sprint se enfoca en el sistema encargado de aplicar las decisiones previamente tomadas y generar la escena final libre de conflictos. Se desarrolla el motor de resolución automática a partir de la información del analizador y visualizador, asegurando coherencia estructural y semántica.

Además, se integra el sistema con [Git](#) para automatizar todo el flujo colaborativo, incluyendo la detección, resolución e integración. Se incorpora un sistema de configuración flexible y protecciones que evitan errores comunes como la reintroducción de conflictos.

4.3.6 Sprint 6: Extensión a Assets

Duración: 2.5 semanas **Semanas:** 17–19

El sexto sprint extiende las capacidades del sistema más allá de escenas [Unity](#) para incluir otros tipos de [assets](#) como [Prefabs](#), materiales y configuraciones. Se implementan detectores especializados y estrategias de resolución adaptadas a las particularidades de cada tipo de [asset](#), ampliando significativamente el alcance y utilidad de la herramienta.

4.3.7 Sprint 7: Integración y Documentación

Duración: 1.5 semanas **Semanas:** 20–21

El sprint final consolida todas las funcionalidades en una interfaz unificada que centraliza el acceso a todas las herramientas desarrolladas. Se implementa un sistema de detección automática que monitoriza continuamente el proyecto y alerta proactivamente sobre conflictos emergentes, junto con un sistema de configuración automática que elimina barreras de entrada para nuevos usuarios. La refactorización del código y la mejora de la documentación de la herramienta.

4.4 Recursos

Entre los recursos utilizados para este proyecto, se han estimado los siguientes teniendo en cuenta su contexto y uso.

4.4.1 Recursos Humanos

Ya que el proyecto se ha realizado de forma individual junto con el tutor, se han definido los siguientes roles dentro del marco Scrum. En la tabla 4.1 se presenta un resumen de las responsabilidades asignadas a cada rol, incluyendo el papel del tutor en el equipo:

Rol	Responsabilidades
Product Owner (Tutor)	Definir los objetivos del proyecto y priorizar las funcionalidades en el <i>product backlog</i> . Supervisar el progreso y aportar feedback estratégico.
Scrum Master	Facilitar el proceso Scrum, identificar y eliminar posibles impedimentos, y promover mejoras continuas en el equipo.
Desarrollador	Diseñar, implementar, probar y documentar cada funcionalidad del sistema, asegurando la calidad y cumplimiento de los requisitos.

Tabla 4.1: Resumen de roles y responsabilidades asumidos durante el desarrollo

4.4.2 Recursos Técnicos

Los recursos técnicos necesarios para el desarrollo de esta herramienta, incluyendo tanto hardware como software especializado, se detallan en la sección 2.2.4 del capítulo de funda-

mentos, donde se describe la selección de tecnologías empleadas para el proyecto.

4.4.3 Costes del Proyecto

El análisis económico del proyecto contempla los costes directos e indirectos durante las 21 semanas planificadas. Para estimar el coste de personal, se han consultado tarifas freelance actualizadas en España.

En el caso del desarrollador freelance, se ha considerado una tarifa media de **40 €/hora**, dentro del rango habitual de **35 a 50 €/hora** según diversas fuentes [36, 37, 38]. Esta cifra resulta más representativa que la referencia inicial de 30 €/hora.

Para el perfil de tutor o gestor técnico (project manager), se ha estimado una tarifa de **50 €/hora**, coherente con los valores habituales entre **35 y 75 €/hora**, con una media de 45–60 €/hora [39, 40, 41].

El desglose completo de los costes puede consultarse en la tabla 4.2.

Concepto	Coste Unitario	Coste Total
Desarrollo (440 horas)		
– Desarrollador (352h × 30 €/h)	30 €/hora	10.560 €
– Tutor / Gestor (88h × 45 €/h)	45 €/hora	3.960 €
Software (Unity Personal, VS Community)	0 €	0 €
Hardware (amortización 6 meses)	—	800 €
Servicios (Internet, electricidad)	100 €/mes	600 €
Documentación y recursos	—	300 €
Total		16.220 €

Tabla 4.2: Desglose de costes del proyecto.

4.5 Planificación Inicial

La planificación inicial del proyecto abarca un total de 21 semanas, distribuidas en siete sprints con duraciones variables, adaptadas a la complejidad técnica de cada fase. En la figura 4.1, se presenta un diagrama de Gantt general que refleja esta distribución temporal y permite visualizar de forma clara las fases principales del desarrollo.

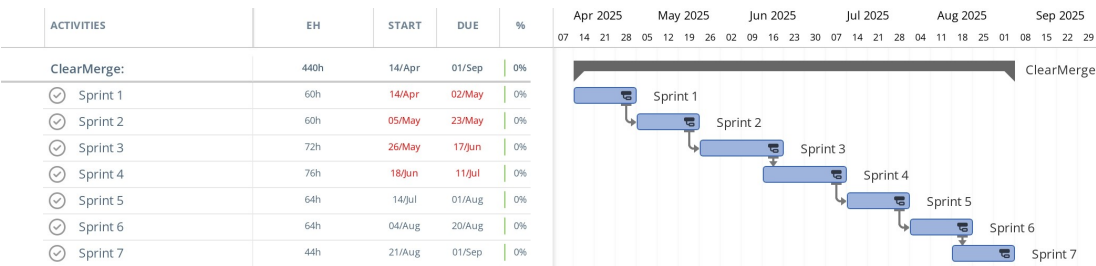


Figura 4.1: Diagrama de Gantt general del proyecto mostrando la distribución de los sprints en el tiempo.

Cada sprint contiene un conjunto específico de actividades que se han planificado en función de sus dependencias, nivel de dificultad y relevancia técnica. En particular, los sprints 3 y 4 cuentan con una mayor asignación de tiempo debido a la complejidad asociada con el desarrollo del núcleo del sistema: la correlación de elementos en conflicto y el sistema de resolución visual.

En la figura 4.2, se muestra un diagrama de Gantt detallado que desglosa las tareas específicas de cada sprint, sus relaciones de dependencia y los hitos técnicos que marcan el avance del proyecto.

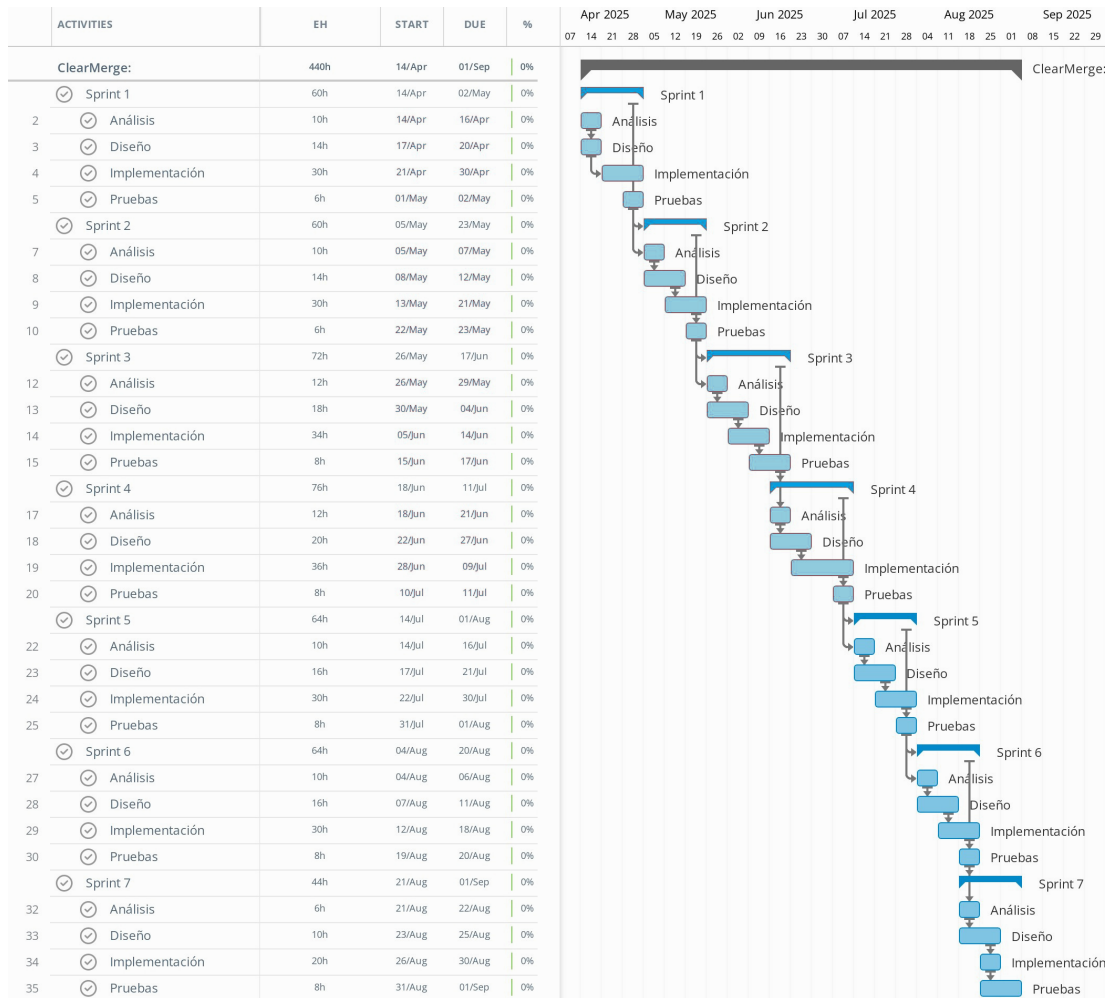


Figura 4.2: Diagrama de Gantt detallado mostrando tareas específicas, dependencias y hitos críticos.

4.6 Resultado del Seguimiento

En base a las estimaciones realizadas, puede observarse que los costes iniciales estimados al inicio del proyecto prácticamente no han sufrido modificación, ya que los tiempos de desarrollo han coincidido de forma muy cercana con lo planificado. Sin embargo, sí se han registrado ajustes puntuales en algunos sprints:

- **Sprint 2:** de 2.5 semanas (18 días) pasó a 1 semana (7 días)
- **Sprint 5:** inicialmente de 2,5 semanas (18 días) se extendió a 3 semanas (21 días) para completar la refactorización del motor de resolución.

- **Sprint 6:** previsto en 2,5 semanas (18 días) se completó en 1,5 semanas (11 días) gracias a la reutilización de la arquitectura de escenas.

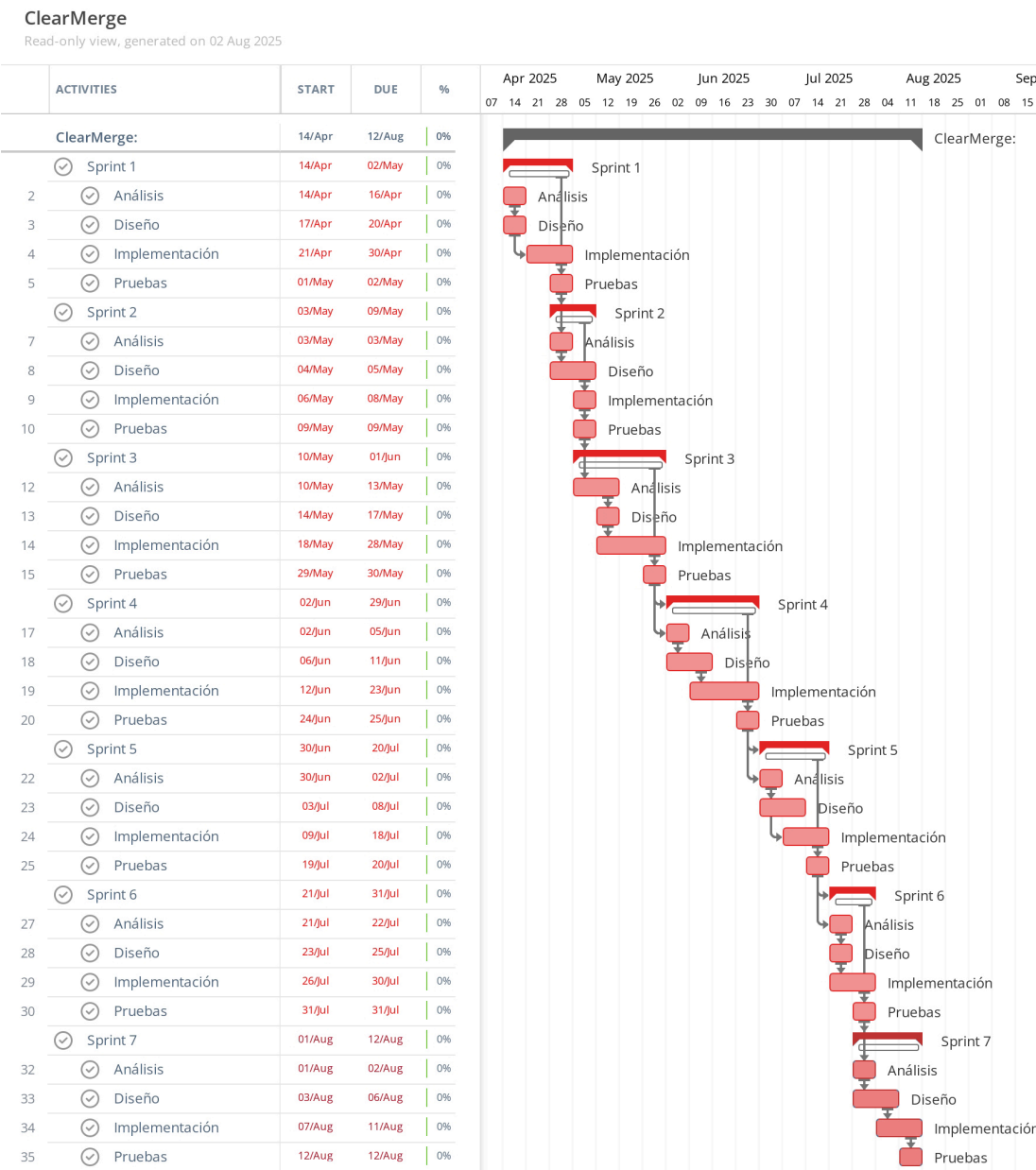


Figura 4.3: Diagrama de Gantt: Planificación Final

Después de estos cambios, la tabla 4.3 resume dichas diferencias e impactos en el desarrollo del proyecto.

Sprint	Duración Estima- da	Duración Real	Desviación (días)	Impacto Económi- co
Sprint 2	18 días	7 días	-11 días	-2.640 €
Sprint 5	18 días	21 días	+3 días	+720 €
Sprint 6	18 días	11 días	-7 días	-1.680 €
Total	—	—	-15 días	-3.600 €

Tabla 4.3: Desviación temporal y económica respecto a la planificación inicial.

Gracias a una ejecución más eficiente en varios sprints, el proyecto finalizó con 15 días de adelanto, lo que se traduce en una reducción de costes directos y una mejora general en la eficiencia del desarrollo.

Capítulo 5

Desarrollo

5.1 Sprint 1: Planificación y Análisis

Durante el primer sprint, nos centraremos en el análisis de los **conflictos** que se producen en **Unity**, con el objetivo de comprender cómo su sistema de archivos **YAML** representa los distintos **GameObjects** dentro de una **escena**. En este sprint no se abordará la implementación de funcionalidades como tal, sino que se sentarán las bases conceptuales y técnicas que permitirán avanzar en los siguientes ciclos de desarrollo.

En la fase de pruebas se analizará un caso real de **conflicto** utilizando el estado por defecto de **Unity**, lo que permitirá identificar los elementos clave que necesitaremos extraer y procesar en sprints posteriores. Esto servirá como referencia para diseñar la arquitectura básica del sistema de análisis automático.

5.1.1 Análisis

En la primera fase se procederá a analizar los requisitos de la herramienta y sus historias de usuario

Requisitos funcionales

Se definieron los requisitos funcionales que debe cumplir la herramienta, que pueden verse en el anexo [A.1](#).

Requisitos no funcionales

En la tabla [5.1](#) se detallan condiciones adicionales relacionadas con compatibilidad, rendimiento y usabilidad que debe cumplir la herramienta:

ID	Nombre	Descripción
RNF-001	Compatibilidad con Unity 6+	Debe funcionar con Unity 6 y versiones posteriores.
RNF-002	Interfaz intuitiva	La interfaz será fácil de usar sin requerir conocimientos técnicos de YAML.
RNF-003	Manejo seguro de archivos	Debe evitar la corrupción de archivos de escena durante su manipulación.
RNF-004	Integración con Git estándar	Compatible con flujos de trabajo Git tradicionales.
RNF-005	Buen rendimiento	Debe ofrecer un rendimiento fluido y respuesta adecuada incluso con escenas de complejidad media.

Tabla 5.1: Requisitos no funcionales del sistema.

Historias de usuario

A partir de los requisitos definidos y con el objetivo de concretar las funcionalidades del sistema desde la perspectiva del usuario, se han redactado las historias de usuario listadas en la tabla 5.2, que servirán de base para la planificación y ejecución de cada sprint.

Tabla 5.2: Historias de usuario del sistema

ID	Descripción	Prioridad
HU-001	Como desarrollador, quiero entender claramente qué conflictos existen en mi proyecto para tomar decisiones informadas	Alta
HU-002	Como desarrollador, quiero tener una interfaz intuitiva para resolver los conflictos de forma eficiente y sin curva de aprendizaje innecesaria	Alta

..... (continúa en la página siguiente)

Tabla 5.2 – (viene de la página anterior)

ID	Descripción	Prioridad
HU-003	Como desarrollador, quiero comparar visualmente las dos versiones de un GameObject conflictivo para entender las diferencias antes de decidir	Alta
HU-004	Como desarrollador Unity , quiero que el sistema detecte automáticamente conflictos en mis escenas para no tener que revisar manualmente cada archivo después de un merge	Alta
HU-005	Como desarrollador, quiero ver qué GameObjects específicos están en conflicto para enfocar mi atención solo en las áreas problemáticas	Alta
HU-006	Como desarrollador, quiero resolver conflictos GameObject por GameObject para mantener control granular sobre las decisiones	Media
HU-007	Como desarrollador, quiero que la resolución sea segura y no corrompa mis archivos de escena	Alta
HU-008	Como desarrollador, quiero resolver conflictos en Prefabs de la misma manera que en GameObjects normales	Baja
HU-009	Como desarrollador, quiero navegar rápidamente a GameObjects conflictivos en escenas complejas usando herramientas de enfoque automático	Media
HU-010	Como desarrollador, quiero una previsualización de mis decisiones de resolución antes de confirmar los cambios definitivos	Media
HU-011	Como desarrollador, quiero que Git se integre automáticamente para hacer commit de mis resoluciones sin pasos manuales adicionales	Media

..... (continúa en la página siguiente)

Tabla 5.2 – (viene de la página anterior)

ID	Descripción	Prioridad
HU-012	Como desarrollador, quiero configurar el comportamiento de la herramienta según mi flujo de trabajo específico	Baja
HU-013	Como desarrollador, quiero detectar y resolver automáticamente conflictos en assets Unity (materiales, texturas, scripts)	Alta
HU-014	Como desarrollador, quiero que los conflictos en archivos .meta se resuelvan automáticamente sin mi intervención	Alta
HU-015	Como desarrollador, quiero ser alertado inmediatamente cuando existan conflictos en mi proyecto al abrir Unity	Media
HU-016	Como desarrollador, quiero una configuración automática inicial que prepare mi proyecto sin configuración manual	Media
HU-017	Como desarrollador, quiero acceder a todas las herramientas de resolución de conflictos desde un punto central unificado	Baja
HU-018	Como desarrollador, quiero ver estadísticas en tiempo real del estado de conflictos en mi proyecto	Baja

5.1.2 Diseño

Como se mencionó anteriormente, este sprint estuvo centrado en concretar los requisitos de la herramienta, así como en identificar las posibles historias que pudieran derivarse de los mismos. A partir del análisis de dichas historias de usuario, fue posible identificar a los dos primeros **actores** que interactuarán con el sistema:

- **Desarrollador:**

Representa al **usuario final** que hará uso de la herramienta para resolver **conflictos** en **escenas** de forma **visual** y **asistida**. Su rol fue definido en base a las necesidades

reflejadas en las historias de usuario, y servirá como **referencia** para el diseño de la **interfaz** y las **funcionalidades** del sistema en los siguientes sprints.

- **Sistema:**

Representa la **herramienta desarrollada** en sí misma. Será responsable de realizar tareas **automatizadas** como la **detección**, **análisis** y **resolución de conflictos**, interactuando con el **Desarrollador** y con el entorno de **Unity/Git**.

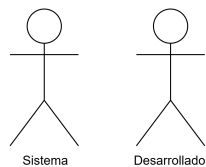


Figura 5.1: Actores identificados

5.1.3 Pruebas

Durante este primer sprint, se plantean pruebas exploratorias centradas en **conflictos** generados específicamente en archivos de **escenas**. Se excluyen por el momento **conflictos** en archivos externos (como **materiales** o **texturas**), con la excepción de los **Prefabs**, que presentan una relación directa con **GameObjects** presentes en la jerarquía de la **escena**.

Uno de los casos más representativos de **conflicto** ocurre cuando dos desarrolladores modifican simultáneamente la posición de un mismo **GameObject** en la **escena** (por ejemplo, un árbol), desde ramas distintas del repositorio. Este escenario genera un **conflicto** directo al realizar la operación de *merge*.



Árbol movido por Desarrollador A



Árbol movido por Desarrollador B

Figura 5.2: Modificaciones paralelas de un mismo **GameObject** por dos desarrolladores

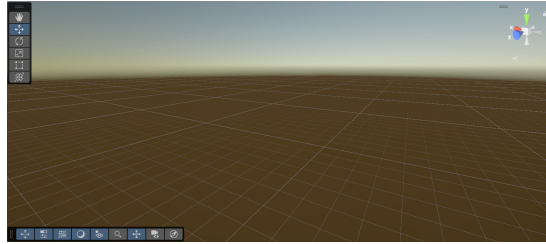


Figura 5.3: Escena con **conflicto** tras la fusión de cambios

Unity, al detectar un **conflicto** de este tipo, responde con las siguientes acciones:

- Activa el *modo seguro* si el **conflicto** existe antes de abrir la **escena**.
- Bloquea la edición completa de la **escena** hasta que el **conflicto** se resuelva.
- Muestra un mensaje de error en la consola, indicando que se requiere intervención manual.

El archivo **YAML** correspondiente a la **escena** contiene los marcadores estándar de **Git**, que permiten observar las diferencias generadas entre ramas. En el listado 5.1, se muestra un fragmento real del **conflicto**, donde puede observarse la discrepancia en la posición del **GameObject**:

```

1  ...
2  --- !u!1 &2222111361972558080
3  GameObject:
4    m_Name: Tree_01
5    ...
6  --- !u!4 &2222111361972585760
7  Transform:
8    m_GameObject: {fileID: 2222111361972558080}
9    ...
10 <<<<<<< HEAD
11   m_LocalPosition: {x: -36.21, y: 1.48, z: -25.55}
12 =====
13   m_LocalPosition: {x: -43.9, y: 1.48, z: -26.2}
14 >>>>>> fad7020d5f610a113596b62cebd1fd168eb7a134
15   ...

```

Listing 5.1: Ejemplo de **conflicto** en archivo **YAML** de **escena**

Aunque este ejemplo es simple, la interpretación de este tipo de archivos puede resultar compleja, incluso para desarrolladores con experiencia, debido al formato jerárquico y técnico de los ficheros ‘.unity’.

Como parte de este sprint, también se ha realizado un análisis preliminar de la estructura interna de estos archivos. Se han identificado los siguientes elementos clave:

- **m_LocalIdentifierInFile**: Identificador único de cada **GameObject** serializado dentro del archivo **YAML**.
- **instanceID**: Identificador temporal de la instancia cargada en memoria.
- **guid**: Identificador global que vincula referencias entre distintos archivos del proyecto.
- Jerarquía de **GameObjects** y sus **componentes** asociados, organizada mediante referencias cruzadas.

Este conocimiento será esencial para la implementación del módulo de análisis semántico de **conflictos**, que se abordará en sprints posteriores.

5.2 Sprint 2: Diseño de Arquitectura

Después del sprint review del anterior sprint, se procede a crear la arquitectura base y patrones que seguirá el proyecto para su desarrollo. Este sprint se centró en establecer las bases estructurales que permitirán un desarrollo modular y escalable en los sprints posteriores.

5.2.1 Análisis

Durante este sprint se organizó y estructuró la información recopilada en el Sprint 1 para poder utilizarla de manera efectiva en el diseño de la arquitectura. Se priorizaron los requisitos funcionales y se establecieron las bases para la modularización del sistema.

El análisis reveló la necesidad de separar las responsabilidades en módulos bien definidos, considerando la complejidad de los archivos **YAML** de **Unity** y la diversidad de elementos que pueden entrar en **conflicto** (escenas, assets, **Prefabs**).

En este sprint no se implementará ninguna historia de usuario, ya que nos centraremos en el diseño del sistema para facilitar implementaciones futuras.

5.2.2 Diseño

Se decidió separar de forma modular los siguientes bloques, para poder así trabajar de forma independiente:

- **Módulo Scenes**: Contiene todas aquellas clases encargadas del análisis, visualización y resolución de **conflictos** en **escenas**.
- **Módulo Assets**: Contiene todas aquellas clases encargadas del análisis, visualización y resolución de **conflictos** en **assets**.

- **Módulo Utils:** Contiene toda la información compartida (constantes y ajustes de usuario), así como utilidades como el setup inicial.

La ventana principal de la herramienta en el editor no está dentro de un módulo ya que es compartida por todos los módulos anteriores.

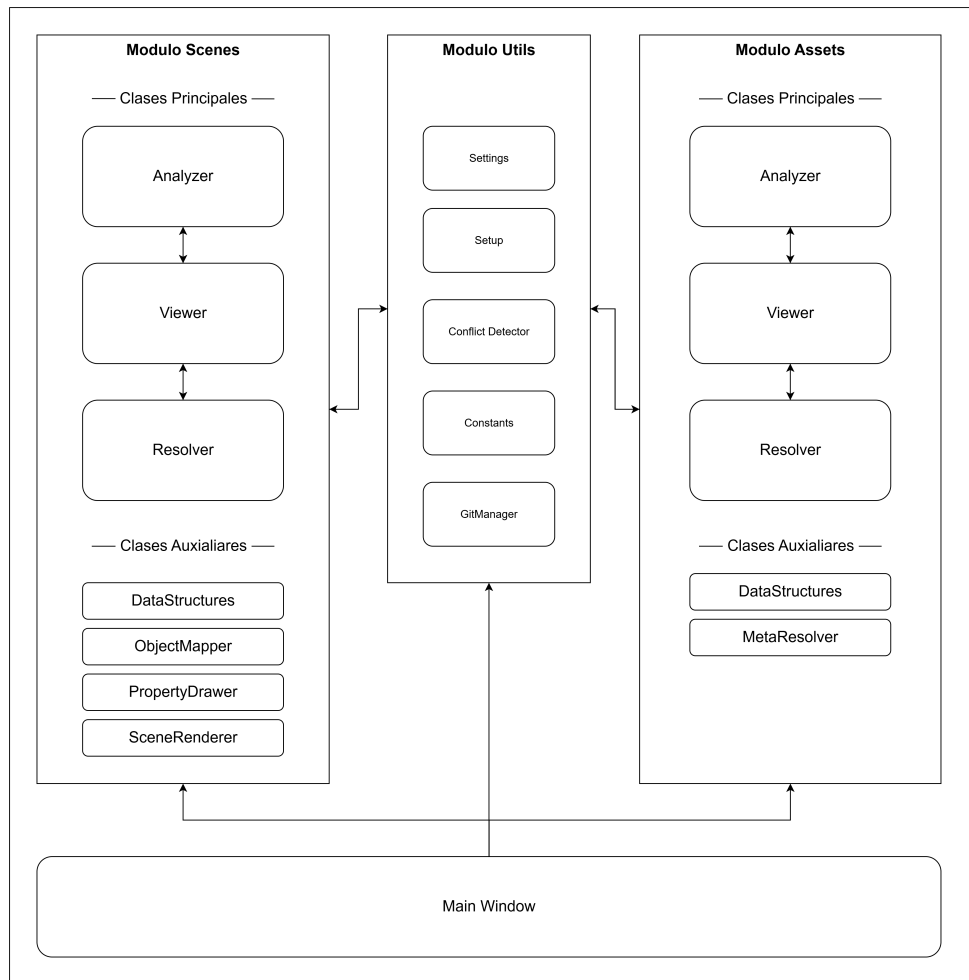


Figura 5.4: Diagrama de la estructura general de clases

Como se observa en la Figura 5.4, los módulos `Assets` y `Scenes` cuentan con tres subsistemas principales, encargados de las fases de análisis, visualización y resolución de **conflictos**. Además, cada uno incluye clases auxiliares específicas que complementan su funcionamiento.

El módulo `Utils` agrupa funcionalidades transversales a toda la herramienta, como:

- Las opciones de configuración general.

- El sistema de inicialización automática (Setup).
- El detector proactivo de **conflictos** dentro del editor de **Unity**.
- Las constantes compartidas entre módulos.
- La clase `GitManager`, que gestiona todas las operaciones relacionadas con **Git**.

Todos estos módulos se integran y son accesibles desde la ventana principal de la herramienta, gestionada por la clase `MainWindow`.

Más adelante se detallarán los componentes internos de cada módulo, así como su funcionamiento particular.

Dentro de esta estructura, se han identificado varios **Patrones de diseño** [42] utilizados para mejorar la modularidad, escalabilidad y mantenibilidad del sistema :

- **Data Transfer Object (DTO):** Se utilizaron estructuras de datos para encapsular datos recurrentes y transferirlos entre subsistemas sin acoplamiento innecesario.
- **Adapter:** Se adaptan estructuras propias de **Unity** (como `GameObject` o `assets`) a estructuras de datos internas más simples y manejables, facilitando así su análisis y posterior resolución.
- **Fachada:** Las ventanas de visualización (*Viewers*) actúan como fachadas, centralizando y simplificando el acceso a los subsistemas de análisis y resolución.
- **Command:** Muchas de las acciones disponibles en los submenús del editor se encapsulan como comandos, permitiendo su ejecución diferida según la interacción del usuario.
- **Observer:** Se emplean eventos del editor de **Unity** para detectar cambios realizados por el usuario o el entorno, lo que permite reaccionar de forma dinámica ante nuevos **conflictos** o modificaciones relevantes.

5.2.3 Implementación

Para la implementación inicial se decidió, como se mencionó antes, separar por módulos, denominados en el entorno de C# como *namespaces*, que permiten aislar el código, todos agrupados dentro del `namespace ClearMerge` (`ClearMerge.Scenes`, `ClearMerge.Assets`, `ClearMerge.Utils`).

La mayoría de clases no heredan de ninguna otra y son estáticas, excepto aquellas que interactúan directamente con el usuario, es decir, las ventanas del editor. Estas heredan de `EditorWindow` y usan la API del editor proporcionada por **Unity**.

5.2.4 Pruebas

En este sprint no se realizaron pruebas funcionales ya que se centró en implementar la arquitectura base que luego completaremos en los próximos sprints. Sin embargo, se verificó la correcta compilación y estructura del código base para asegurar que los módulos están correctamente definidos y que las dependencias entre ellos son las esperadas.

5.3 Sprint 3: Analizador de conflictos

Antes de comenzar el Sprint 3, se revisó el *Product Backlog* y se priorizó la historia de usuario **HU-001**, centrada en la detección clara de **conflictos** en **escenas**.

En este sprint se abordó la implementación del analizador de **conflictos** para **escenas**, componente esencial para detectar y comprender discrepancias surgidas durante fusiones de ramas en **Git**. El objetivo fue dotar al sistema de la capacidad de identificar **GameObjects** afectados, clasificar los **conflictos** y preparar la base para futuras herramientas de resolución.

5.3.1 Análisis

Se determinó que los **conflictos** deben ser interpretados no sólo a nivel textual (marcadores de **Git**), sino también semántico, debido al formato **YAML** que utiliza **Unity**, para poder entender a fondo los **conflictos** y a que **componentes** afectan.

Se priorizó la siguiente historia de usuario:

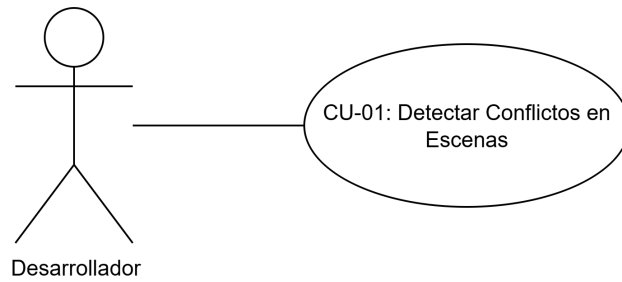
- **HU-001:** Como desarrollador, quiero entender claramente qué **conflictos** existen en mi proyecto para tomar decisiones informadas.

De esta historia surge el primer caso de uso:

CU-01: Detectar conflictos en escenas

- **Descripción:** El desarrollador necesita identificar automáticamente **conflictos** en archivos de **escena Unity**.
- **Precondiciones:** Existe un **conflicto Git** en archivos `.scene`.

La Figura 5.5 representa gráficamente este caso de uso y su relación con el actor principal, el desarrollador. Los casos de uso se mostraran de manera detallada en las tablas del apéndice A.2, con todas sus precondiciones, postcondiciones y flujo.

Figura 5.5: Caso de uso CU-01: Detectar **conflictos** en **escenas**

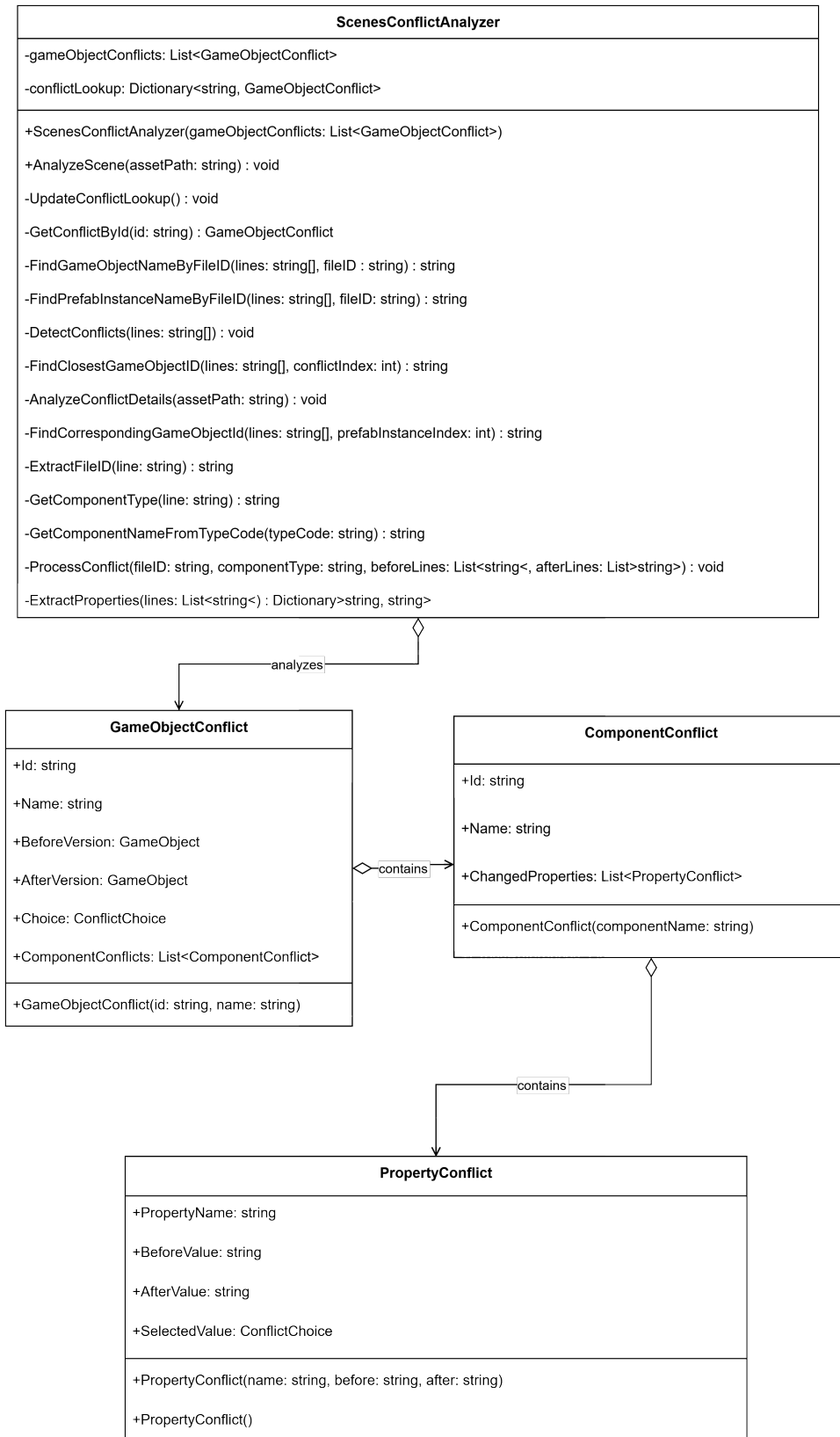
5.3.2 Diseño

A partir de este sprint se establece una nomenclatura estándar para referirse a las versiones implicadas en un **conflicto**: **BEFORE** representa la versión previa al merge (HEAD en Git) y **AFTER** la versión entrante (INCOMING). Esta terminología se adopta para facilitar la comprensión al usuario, así como mantener una abstracción respecto al sistema de control de versiones subyacente, facilitando una posible compatibilidad futura con otras plataformas.

El análisis de **conflictos** se estructura como un proceso en varias etapas:

1. Lectura del archivo de **escena** en formato **YAML**.
2. Detección de bloques de **conflicto** generados por Git (<<<<<<, =====, >>>>>>).
3. Análisis sintáctico y semántico de los **GameObjects** y **componentes** implicados.
4. Construcción de estructuras de datos que representen de forma organizada cada **conflicto** detectado.

La Figura 5.6 muestra el modelo de clases diseñado para representar los **conflictos** de forma estructurada.



Estructuras de datos principales

En busca de abstraer el problema se crearon estructuras de datos para el **Módulo Scenes**, siguiendo así el **Patron DTO**:

- **GameObjectConflict**: representa un `GameObject` con **conflictos** detectados, incluyendo su nombre y **componentes** afectados.
- **ComponentConflict**: modela un **componente** de **Unity** en **conflicto** (por ejemplo, `Transform`), conteniendo una lista de propiedades modificadas.
- **PropertyConflict**: contiene los valores en **conflicto** para una propiedad específica, diferenciando entre la versión **BEFORE** y **AFTER**.

Este modelo de datos permite encapsular los **conflictos** de forma jerárquica, facilitando tanto la visualización como la posterior resolución interactiva en el editor.

5.3.3 Implementación

Se implementó la clase `ConflictAnalyzer` como núcleo del módulo `ClearMerge.Scenes`, delegando cada paso del análisis en métodos especializados.

ConflictAnalyzer

Encargada del análisis completo del archivo de **escena**. Internamente, divide su lógica en varios métodos clave:

- `AnalyzeScene()`: método principal que prepara la lista de **conflictos**, lanza el análisis y muestra los resultados por consola.
- `DetectAndAnalyzeConflicts()`: recorre el archivo de **escena**, detecta los bloques de **conflicto Git** y llama a los procesadores internos.
- `ProcessConflict()`: compara propiedades **BEFORE/AFTER** y crea las instancias de `ComponentConflict` y `PropertyConflict`.
- `ExtractProperties()`: convierte líneas **YAML** en un diccionario clave-valor con todas las propiedades relevantes.

GameObjectConflict

Modelo que encapsula la información por **GameObject** conflictivo:

- **Id**: identificador local extraído del **YAML** (`m_LocalIdentifierInFile`).

- Name: nombre del `GameObject` en la jerarquía de `Unity`.
- `ComponentConflicts`: lista de instancias de `ComponentConflict` relacionadas con este `GameObject`.

ComponentConflict y PropertyConflict

Estas clases modelan el detalle de cada `conflicto`:

- **ComponentConflict**
 - Name: tipo de `componente Unity` afectado (ej. `Transform`, `Light`, etc.).
 - `ChangedProperties`: lista de propiedades modificadas entre ambas versiones.
- **PropertyConflict**
 - `BeforeValue`: valor de la propiedad antes del merge.
 - `AfterValue`: valor de la propiedad en la versión entrante.

Análisis de prefabs

Durante el desarrollo del sistema de visualización de `conflictos`, se identificó un caso particular en el tratamiento de `GameObjects` instanciados desde `Prefabs`. Estos `GameObjects`, a diferencia de los elementos definidos directamente en la `escena`, no siempre poseen un campo `m_LocalIdentifierInFile` confiable o persistente para su localización directa.

Para resolver esta limitación, se diseñó una estrategia alternativa basada en el uso del identificador `GlobalObjectId`, una estructura proporcionada por `Unity` que permite identificar de forma única cualquier `GameObject` en el proyecto, incluso a través de `escenas` y referencias indirectas.

El formato de representación de un `GlobalObjectId` es:

$$\text{GlobalObjectId_V1} - \{i\} - \{a\} - \{l\} - \{p\}$$

donde:

- `{i}`: Tipo de `GameObject` (0 = Null, 1 = Asset importado, 2 = gameobject en `escena`, 3 = Asset fuente).
- `{a}`: GUID del asset (identificador global único asignado por `Unity` a cada asset).
- `{l}`: Identificador local del `GameObject` dentro del archivo.

- {p}: Identificador local de la instancia del **Prefab**. Si no pertenece a un **Prefab**, este valor es 0.

Esta estructura permitió localizar con precisión **GameObjects** instanciados desde **Prefabs**, y se implementó un nuevo campo en la clase `GameObjectConflict` para almacenar el `GlobalID`, y un nuevo método `GetGameObjectIds()` en `ConflictAnalyzer`. Sin embargo, su uso implica una mayor carga computacional debido a las operaciones de mapeo indirecto requeridas, por lo que se decidió restringir esta técnica exclusivamente a **GameObjects** provenientes de **Prefabs**. Para el resto de **GameObjects** de *escena*, se mantiene el uso prioritario del campo `m_LocalIdentifierInFile`, que resulta mucho más eficiente en cuanto a rendimiento.

5.3.4 Pruebas

Se replicó el escenario de **conflicto** descrito y probado durante el Sprint 1, en el cual dos desarrolladores modifican simultáneamente la posición de un **GameObject** tipo árbol desde ramas distintas del repositorio, generando un **conflicto** directo en su **componente Transform**.

El contenido conflictivo fue el mismo mostrado en el listado 5.1, donde se observa claramente la discrepancia entre versiones en la propiedad `m_LocalPosition` en las líneas 11–14.

El analizador detectó correctamente este **conflicto**, generando el listing de la fig 5.2:

```
1 Conflict ID: 56781231, Name: Tree
2 Component: Transform
3 Property: m_LocalRotation.x
4 Before: 0           After: 0.707
5 Property: m_LocalRotation.w
6 Before: 1           After: 0.707
```

Listing 5.2: Log generado por el analizador de **conflictos**

Adicionalmente, se probaron **conflictos** con instancias de **Prefabs**. Inicialmente no fueron detectados, pero gracias al uso de `GlobalObjectId` se logró su localización en *escena* y funcionamiento, tal y como cualquier **GameObject**.

La Figura 5.7 muestra el flujo completo del `ConflictAnalyzer`, ahora implementado los **Prefabs** en el flujo de análisis.

Al finalizar el Sprint 3, se realizó un *Sprint Review* donde se validó la capacidad del analizador para detectar y clasificar **conflictos**, y se ajustaron las prioridades del *Product Backlog* para el siguiente sprint.

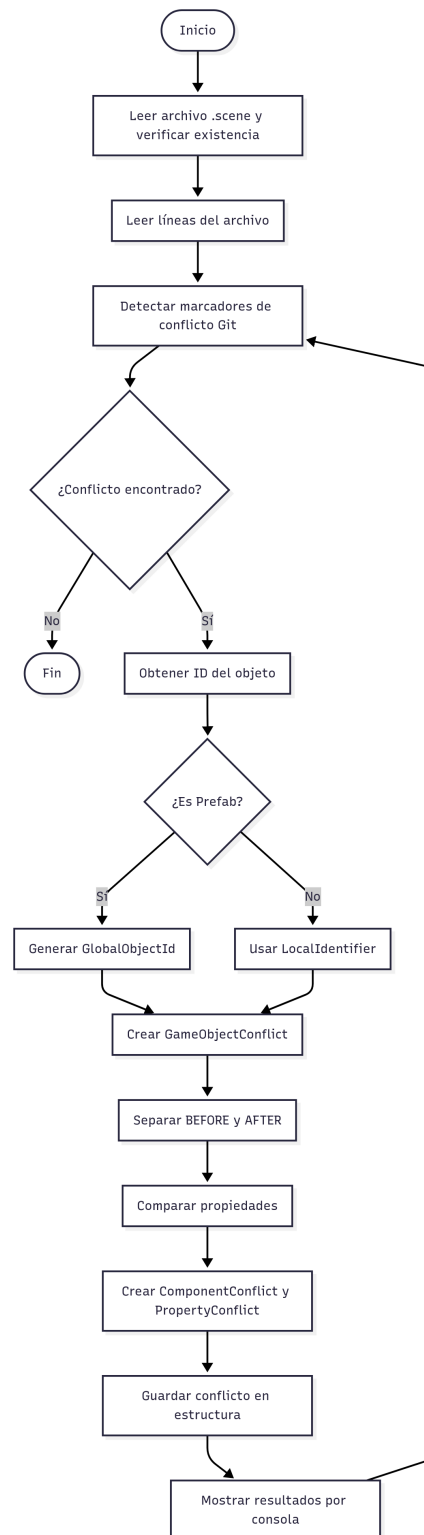


Figura 5.7: Flujo de análisis del ConflictAnalyzer

5.4 Sprint 4: Visualizador de Conflictos

Antes del Sprint 4, se revisó el *Product Backlog* y se seleccionaron historias de usuario relacionadas con la visualización y resolución intuitiva de **conflictos**.

En este sprint se abordó el diseño e implementación del sistema de visualización de **conflictos** en **escenas Unity**, centrado en facilitar al desarrollador la inspección y resolución de **conflictos** detectados en archivos de **escena**. El objetivo principal fue permitir la comparación visual entre versiones de **GameObjects**, la selección interactiva de resoluciones y la navegación eficaz dentro del editor de **Unity**, todo desde una interfaz integrada y sin abandonar el flujo de trabajo del desarrollador.

5.4.1 Análisis

Durante el análisis se priorizaron las siguientes historias de usuario:

- **HU-002:** Interfaz intuitiva para resolver **conflictos**.
- **HU-003:** Comparación visual entre versiones de un **GameObject**.
- **HU-005:** Identificación precisa de **GameObjects** en **conflicto**.
- **HU-006:** Resolución granular **GameObject** por **GameObject**.
- **HU-009:** Navegación visual rápida y precisa.

A partir de estas historias, se definieron los siguientes casos de uso en la figura 5.8:

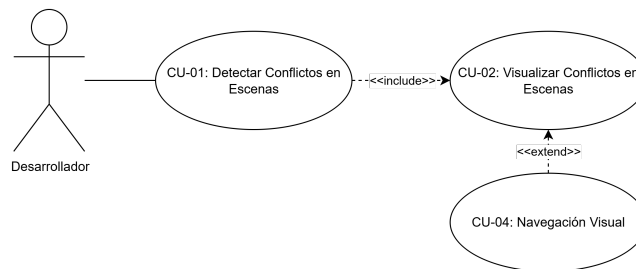


Figura 5.8: Casos de uso: Visualización y navegación de **conflictos**

CU-02: Visualizar conflictos en escenas

- **Descripción** : Comparar visualmente las versiones en **conflicto** de cada **GameObject**
- **Precondición**: Existen **conflictos** previamente detectados.

CU-03: Navegación Visual (extiende CU-02)

- **Description** : Controlar herramientas de navegación y enfoque dentro de la vista de resolución.

El sistema de resolución se pensó para ser abierto directamente dentro de una **escena** en **conflicto**. Al cargarlo, escanea automáticamente el archivo de **escena** y genera una lista detallada con los **GameObjects** afectados, los **componentes** involucrados y las propiedades con diferencias entre versiones.

Tras ello, el usuario puede iniciar la previsualización que carga ambas versiones de la **escena** (**BEFORE** y **AFTER**) en modo aditivo. Inicialmente se probó activar ambas simultáneamente para ver todos los cambios visualmente, pero esto provocaba superposición, **conflictos** de iluminación, pérdida de contexto jerárquico y problemas de rendimiento.

Se evaluaron soluciones alternativas, como desactivar **GameObjects** no conflictivos o eliminar padres jerárquicos, pero se descartaron por la pérdida de contexto estructural. Dado que **Unity** no permite vistas múltiples de **escena**, se optó por un enfoque de **cambio activo**: se mantiene solo una **escena** activa en cada momento, permitiendo alternar entre versiones para analizar los cambios.

5.4.2 Diseño

El sistema se diseñó en torno a la clase principal **ConflictViewerWindow**, que, junto con un conjunto de clases auxiliares, permite crear una ventana personalizada dentro del editor de **Unity**. Esta interfaz lista los **conflictos** detectados y proporciona mecanismos interactivos para navegar entre ellos de forma visual.

En la Figura 5.9 se muestran el diagrama de clases para este sprint:

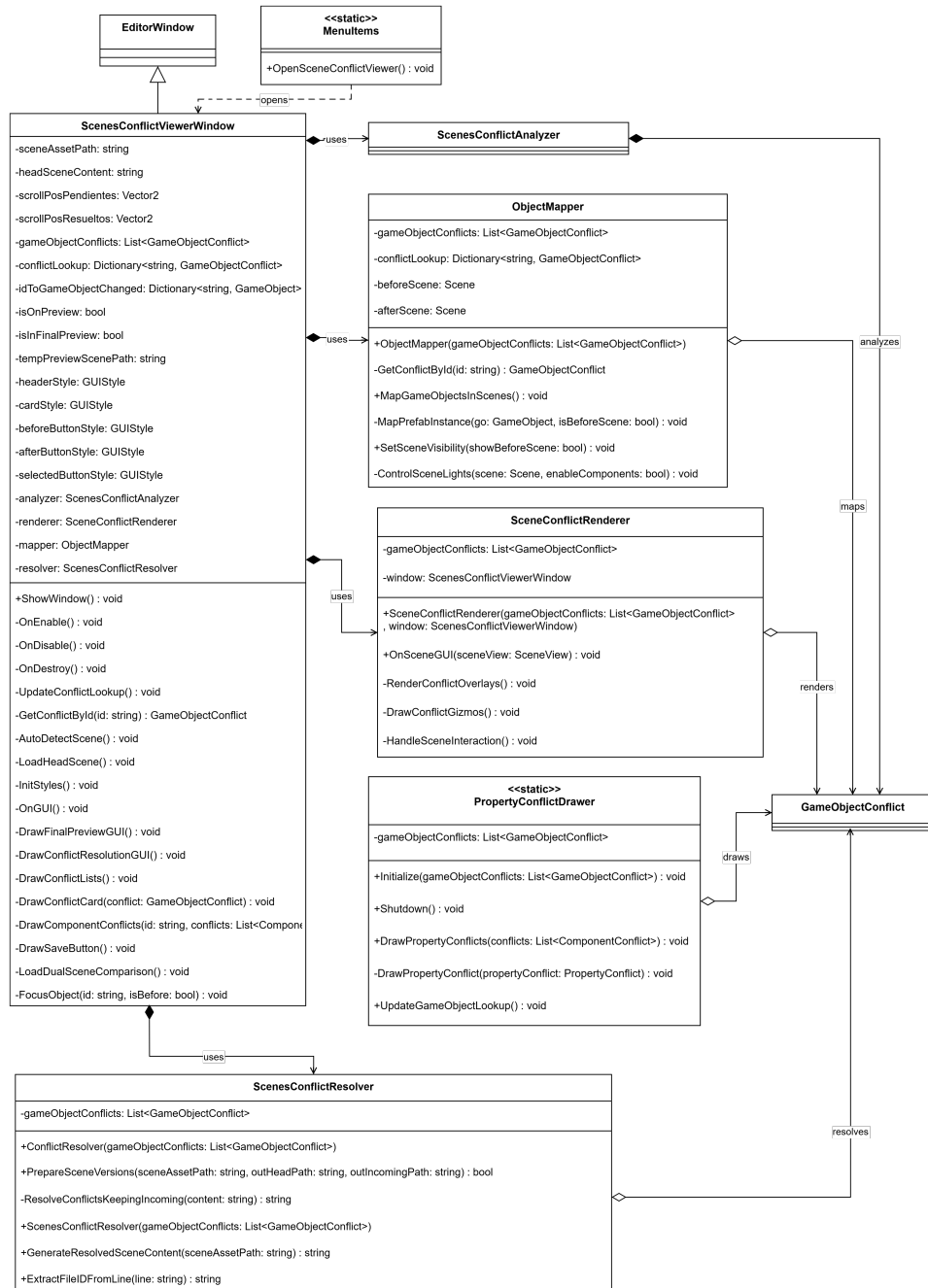


Figura 5.9: Diagrama de clases del sistema de visualización de conflictos

Entre los componentes clave se encuentran:

- **ConflictViewerWindow**: interfaz principal de usuario.
- **ConflictResolver**: convierte los conflictos en dos escenas

- **MenuItems**: define accesos en la barra de menú superior.
- **ObjectMapper**: enlaza identificadores con `GameObjects`.
- **SceneConflictRenderer**: dibuja visualmente los `GameObjects` en escena.
- **PropertyConflictDrawer**: integra la información en el inspector.

Interfaz de usuario

La clase principal es `ConflictViewerWindow`, una extensión de `EditorWindow` que presenta la interfaz principal del sistema. Al abrirse, esta ventana escanea automáticamente los `conflictos` existentes en la `escena` activa y los presenta en una lista con los `GameObjects` afectados, los `componentes` involucrados y las propiedades en disputa.

Para abrir esta ventana se ha utilizado la funcionalidad `MenuItem` proporcionada por `Unity`, lo cual permite registrar accesos personalizados en la barra superior del editor. La clase `MenuItems` define esta entrada bajo el nombre `Scene Conflict Viewer`, accesible desde el menú desplegable `Tools > ClearMerge`, como se muestra en la Figura 5.10.

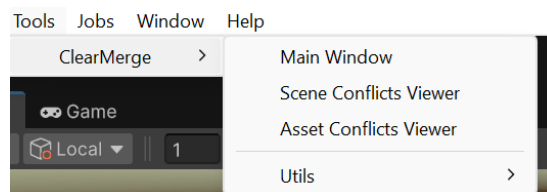


Figura 5.10: Acceso a la herramienta desde el menú `Tools > ClearMerge`

Una vez abierto el visor, se muestra una interfaz con la lista de `GameObjects` en `conflicto` y los detalles de sus diferencias. La Figura 5.11 presenta esta ventana justo después del escaneo inicial de `conflictos`.

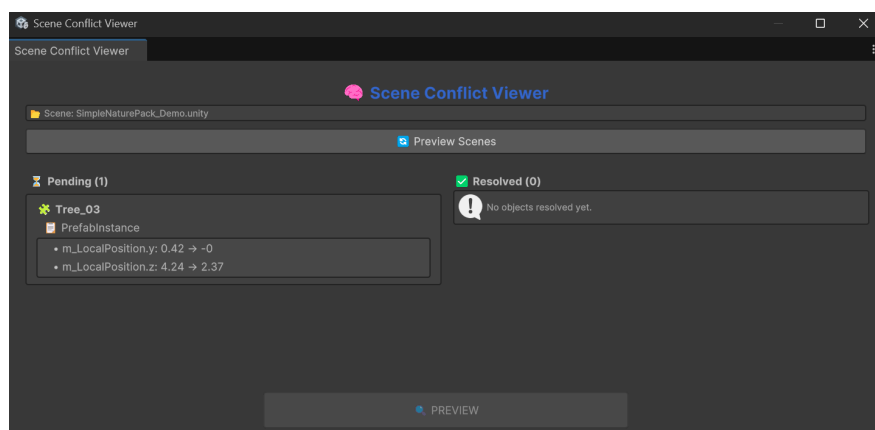


Figura 5.11: Ventana Conflict Viewer tras escaneo inicial de `conflictos`

Al pulsar el botón previsualización la herramienta separa los **conflictos** basándose en la versión a la que pertenecen (**BEFORE** o **AFTER**), tal como ya hizo el analizador durante el Sprint 3, para luego ser juntados de nuevo en dos **escenas** separadas por la clase `ConflictResolver`. A continuación carga ambas versiones de la **escena** de forma aditiva para que el usuario pueda compararlas visualmente y alternar entre ellas sin perder el contexto jerárquico.

Además, se añaden botones de enfoque visual, marcados en amarillo y rojo y etiquetados como **FOCUS**, que centran automáticamente la cámara en el `GameObject` correspondiente, facilitando su localización dentro de **escenas** complejas. Finalmente, se incorporan controles globales para aplicar resoluciones en bloque. La Figura 5.12 ilustra esta versión extendida de la interfaz una vez cargadas las **escenas**.

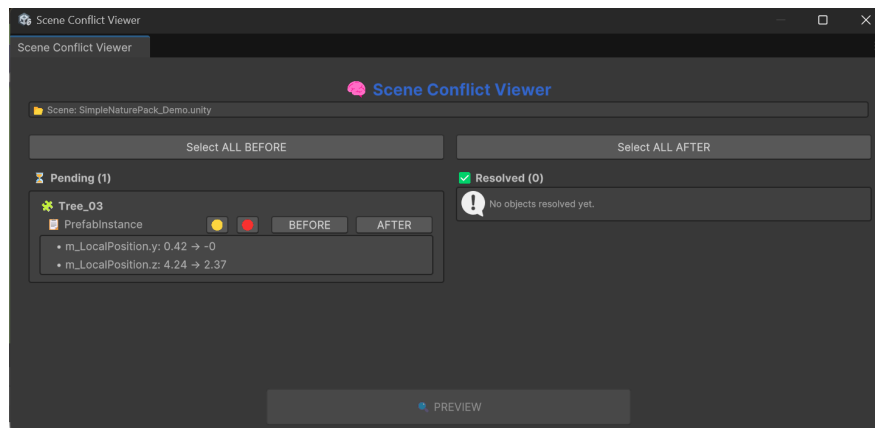


Figura 5.12: Ventana Conflict Viewer con **escenas** cargadas y controles activos

En el editor de jerarquía (Figura 5.13), se visualiza cómo ambas **escenas** han sido cargadas de forma aditiva, estas se guardan de forma temporal en una carpeta en el directorio `Assets/Editor/ClearMerge/Temp`, con sus respectivos nombres identificativos. En lugar de mantener ambas **escenas** activas simultáneamente, como se mencionó anteriormente, se optó por un sistema de **cambio activo**: se carga una versión (**BEFORE** o **AFTER**) y se permite al desarrollador alternar entre ellas.

Se puede observar por los ojos tachados a la izquierda, que solo esta activa la **escena** `TempBeforeMerge`, lo cual permite preservar el contexto jerárquico sin interferencias visuales ni duplicación de `GameObjects` estáticos o luces.

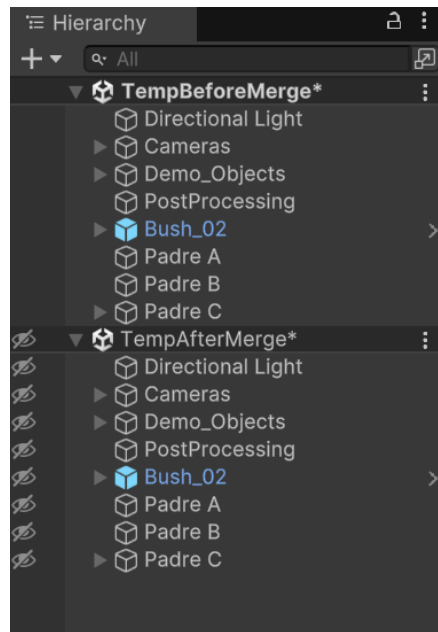


Figura 5.13: Jerarquía tras cargar ambas **escenas** de forma aditiva (**BEFORE** y **AFTER**)

Visualización en escenas

Para representar visualmente los **conflictos** en la ventana de **escena**, se diseñó un sistema marcadores.

En la Figura 5.14 puede observarse el resultado de esta estrategia. En (1) se muestra la **escena** en su versión **AFTER**, y en (2) la **escena BEFORE**. En ambos casos se emplean marcadores visuales, implementados mediante la clase `SceneConflictRenderer`, para indicar con precisión la posición del **GameObject** conflictivo. Estos cambian según nuestras selecciones para dar información rápida, para ver si están resueltos los **conflictos** de ese **GameObject** de forma visual.



(1) **escena AFTER**



(2) **escena BEFORE**

Figura 5.14: Visualización de ambas **escenas** cargadas: **AFTER** (1) y **BEFORE** (2)

Visualización de propiedades en el Inspector

Además de la visualización en la ventana de **escena**, se integró un visor en el Inspector de **Unity** para mostrar con claridad las propiedades de cada **componente** afectado por un **conflicto**. Este visor, proporcionado por la clase `PropertyConflictDrawer`, añade un encabezado adicional cuando se selecciona un **GameObject** conflictivo, mostrando en detalle los valores **BEFORE** y **AFTER** de cada **componente** como se ve en la Figura 5.15.

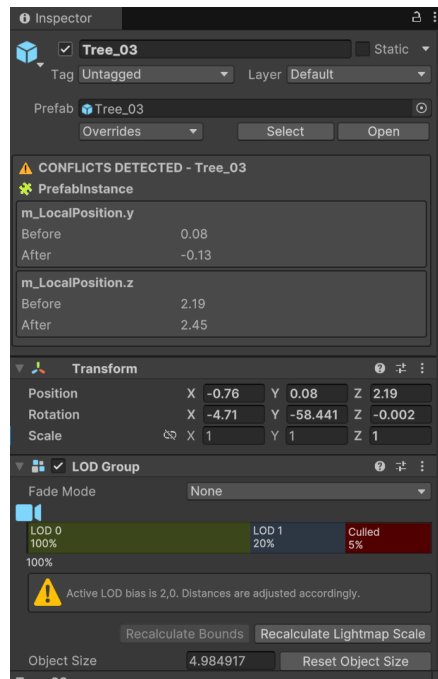


Figura 5.15: Vista en el Inspector del gameobject conflictivo seleccionado

5.4.3 Implementación

A continuación se describen las clases principales desarrolladas.

ConflictViewerWindow

Clase principal de la interfaz. Permite visualizar, comparar, resolver y previsualizar **conflictos** entre dos versiones de una **escena Unity**.

- `ShowWindow()`: Abre la ventana del visor de **conflictos**.
- `OnGUI()`: Dibuja la interfaz gráfica de usuario.
- `LoadDualSceneComparison()`: Carga las versiones **BEFORE** y **AFTER** para su comparación visual.

ConflictResolver

Esta clase aplica las decisiones del usuario y genera el contenido de la nueva **escena** resultante con los **conflictos** resueltos.

- `GenerateResolvedSceneContent()`: Devuelve el contenido textual de la **escena** unificada según las elecciones de cada **conflicto**.
- `PrepareSceneVersions()`: Prepara las versiones temporales de **escena** para ser comparadas visualmente.

MenuItems

Clase estática con accesos desde el menú superior de **Unity**.

- `OpenConflictViewer()`: Abre la ventana de resolución de **escenas**.

ObjectMapper

Asocia los identificadores de **conflictos** con instancias reales de **GameObjects** en las **escenas BEFORE** y **AFTER**, y controla su visibilidad.

- `MapGameObjectsInScenes()`: Mapea todos los **GameObjects** conflictivos a sus versiones reales en ambas **escenas** cargadas.
- `SetSceneVisibility()`: Muestra u oculta la **escena BEFORE** o **AFTER**, incluyendo luces, audio y efectos..

SceneConflictRenderer

Responsable de mostrar en la vista de **escena** marcadores visuales para cada **GameObject** en **conflicto**.

- `OnSceneGUI()`: Dibuja marcadores y etiquetas en la **escena** que representan las versiones en **conflicto**.
- `FocusObjectInScene()`: Permite centrar la vista en un **GameObject** al hacer clic en su marcador.

PropertyConflictDrawer

Extiende dinámicamente el Inspector de **Unity** para mostrar de forma clara las propiedades que presentan **conflictos**.

- `DrawConflictData()`: Añade información visual de las propiedades conflictivas debajo del Inspector nativo.

5.4.4 Pruebas

Las pruebas de este sprint se centraron en validar el flujo completo del sistema, desde la apertura inicial de la ventana `ConflictViewerWindow` hasta la generación final de la *escena* unificada con las resoluciones aplicadas por el usuario. Se verificó que los `GameObjects` en *conflicto* aparecían correctamente listados, con sus *componentes* y propiedades. Véase Figura 5.11.

Se comprobó que los marcadores en *escena* señalaban con precisión cada `GameObject`, incluso en *escenas* desactivadas (5.14).

Los botones **BEFORE**, **AFTER** y **Focus** funcionaron como previsto para alternar versiones y centrar la vista (5.12).

El Inspector mostró correctamente las propiedades en *conflicto* mediante `PropertyConflictDrawer` (Figura 5.15).

El *conflicto* utilizado como caso de prueba principal fue el del `GameObject Tree_03`, ya introducido en el Sprint 2 y documentado en el código `YAML` de la Figura 5.1. A partir del identificador de dicho *conflicto*, se localizó el `GameObject` correspondiente, se visualizó su posición en la *escena* y se inspeccionaron las propiedades modificadas directamente en el inspector mediante la herramienta desarrollada.

5.5 Sprint 5: Sistema de Resolución

Al iniciar el Sprint 5, se revisó el *Product Backlog* para incluir historias de usuario sobre la generación de la *escena* final y la integración con `Git`. Este sprint abordó el desarrollo de la etapa final del flujo de resolución: la generación de una nueva *escena* consolidada a partir de las elecciones del desarrollador, la integración automática con el sistema de control de versiones `Git`, y la incorporación de un sistema de configuración persistente y personalizable.

5.5.1 Análisis

Se desarrollaron funcionalidades vinculadas a las siguientes historias de usuario:

- **HU-007:** Seguridad en el proceso de resolución.
- **HU-008:** Soporte para *Prefabs*.
- **HU-010:** Previsualización antes de confirmar cambios.
- **HU-011:** Commit automático en `Git` tras resolución.
- **HU-012:** Configuración personalizable de la herramienta.

Los casos de uso derivados se ilustran en la Figura 5.16:

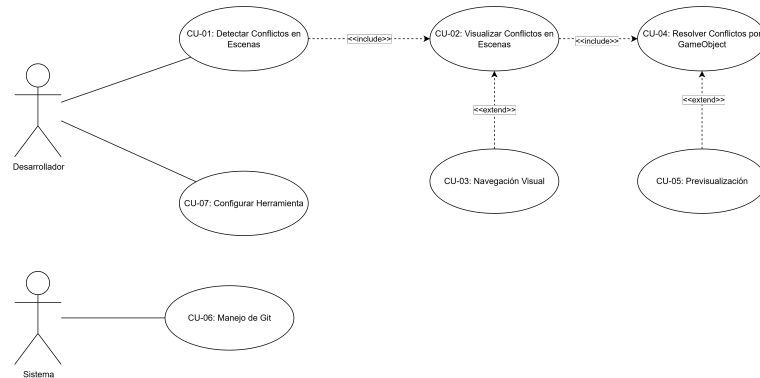


Figura 5.16: Casos de uso principales del Sprint 5

CU-04: Resolver Conflictos por GameObject

- **Descripción:** Permite al desarrollador resolver conflictos seleccionando versiones por objeto.
- **Precondición:** Se está visualizando el [conflicto](#).

CU-05: Previsualización (extiende CU-03)

- **Descripción:** Ver el resultado final antes de aplicar una resolución.
- **Precondición:** Se ha seleccionado una versión.

CU-06: Manejo de Git

- **Descripción:** Ejecuta las diversas operaciones y comandos con el sistema de [Git](#).

CU-07: Configurar Herramienta

- **Descripción:** Permite definir parámetros de resolución y preferencias del usuario.
- **Precondición:** Herramienta instalada.

5.5.2 Diseño

La clase `ConflictResolver`, creada y descrita en el Sprint 4, se reutiliza aquí para generar la [escena](#) final a partir de las decisiones del usuario, cerrando así el ciclo completo de resolución.

Durante la resolución, se recorren todos los `GameObjects` en `conflicto` y, según la versión seleccionada (**BEFORE** o **AFTER**), se clonan dentro de una nueva `escena` vacía. También se copian los `GameObjects` no conflictivos para garantizar la consistencia del entorno.

La estrategia de guardado depende de la configuración establecida por el usuario:

- Si se elige sobrescribir, la nueva `escena` reemplaza directamente la original.
- Si se opta por conservar la original, la `escena` resultante se guarda con un sufijo `_merged`, tal como está implementado en el método `GenerateResolvedScene()`, donde se concatena el nombre original con dicho sufijo para crear un nuevo archivo.

Previsualización Final

Una vez que el usuario ha resuelto todos los `conflictos` y no queda ningún `GameObject` pendiente por decidir, se habilita automáticamente el botón de previsualización. Al pulsarlo, la interfaz cambia por completo, como se muestra en la Figura 5.17, y se genera una nueva `escena` provisional donde se aplican las decisiones tomadas.

En esta `escena` previsualizada, el desarrollador puede modificar libremente tanto `GameObjects` conflictivos como elementos no afectados, para ajustar manualmente la `escena` antes de confirmarla. Una vez terminada la edición, el usuario puede confirmar y guardar la `escena` final, ejecutando la estrategia configurada en `Git`, o volver atrás para cambiar decisiones previas

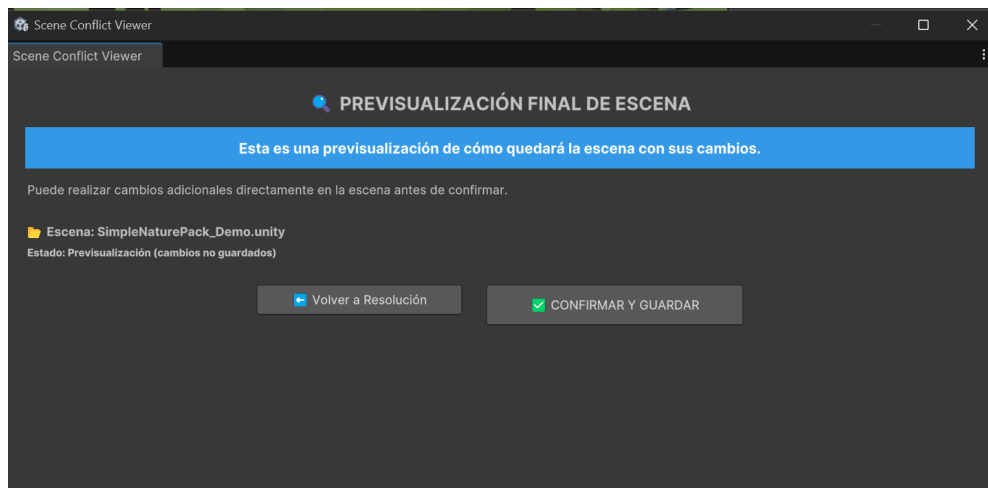


Figura 5.17: Vista de la interfaz durante la previsualización final

Automatización de Git: GitManager

Para asegurar un flujo de trabajo continuo e integrado, se implementó el soporte automático para `Git` mediante la clase `GitManager`, ubicada en el módulo `ClearMerge.Utils`.

Esta clase estática encapsula todas las operaciones relevantes con el sistema de control de versiones, y es la encargada de reincorporar al repositorio la *escena* una vez resueltos todos los *conflictos*.

Entre sus responsabilidades destacan:

- Detección automática de la rama actual.
- Ejecución de commits automáticos tras confirmar la resolución.
- Push automático al repositorio si se ha activado dicha opción.
- Implementación de una regla de protección pre-push que impide subir archivos con marcadores de *conflicto* visibles (como los símbolos <<<< y >>>> presentes en archivos *YAML* sin resolver).

Este último punto es especialmente crítico para evitar la corrupción de *escenas Unity* en entornos colaborativos.

Con esta última clase se completa el sistema de resolución de *conflictos* en *escenas*. La Figura 5.18 muestra el diagrama de clases conjunto que abarca el módulo *Scenes* y las utilidades principales de *Utils*, como *GitManager* y *MenuItem*s. Las constantes más comunes se han centralizado en *ClearMergeConstants*.

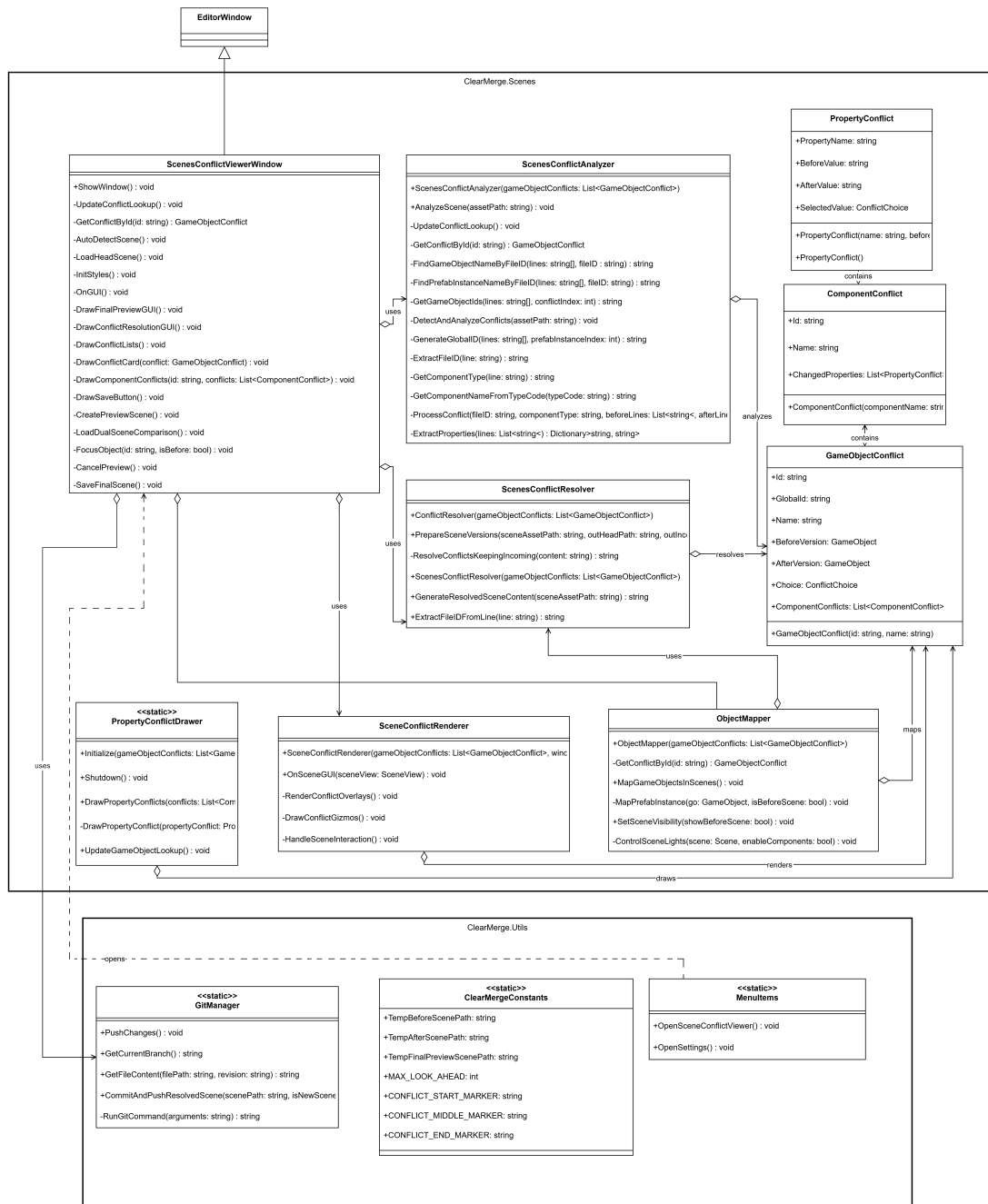


Figura 5.18: Diagrama general del sistema de resolución de conflictos

Configuración Persistente: Settings Provider

Con el objetivo de ofrecer flexibilidad al usuario, se desarrolló un panel de configuración personalizado utilizando la API `SettingsProvider` de **Unity**. Este panel se integra dentro de **Project Settings** como se ve en la figura 5.19 y permite ajustar tanto el comportamiento

del sistema de resolución como su interacción con **Git**. También se añadió un acceso directo a la sección desde la clase `MenuItem`s.

Los parámetros configurables incluyen:

- **Estrategia de resolución:** sobrescribir la **escena** original o guardar una nueva con sufijo.
- **Modo de commit:** ejecutar automáticamente un commit en la rama actual, crear una nueva rama, o desactivar la funcionalidad.
- **Modo seguro Git:** bloquea cualquier push si hay archivos **YAML** con **conflictos** no resueltos.
- **Filtrado de `GameObjects`:** posibilidad de desactivar tipos especiales como luces o cámaras durante la previsualización.
- **Mapeo de componentes:** permite asociar nombres personalizados a **componentes** no estándar.

El sistema guarda estas preferencias de forma persistente en el proyecto, asegurando que se respeten entre sesiones.

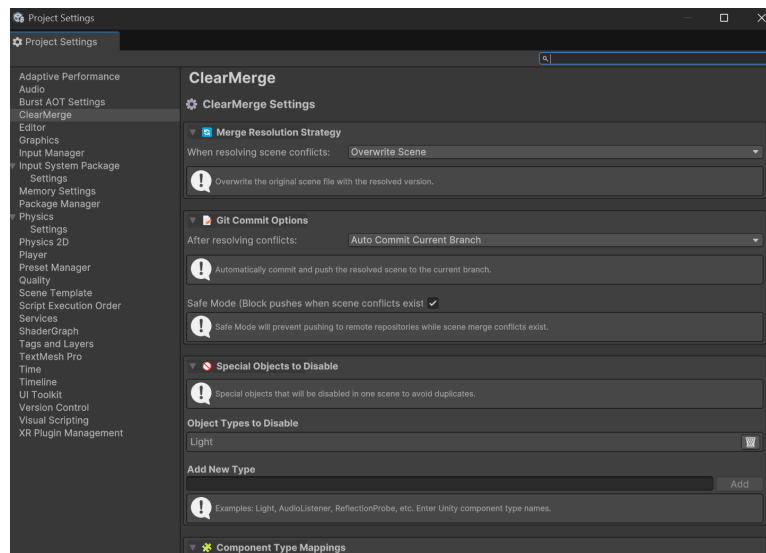


Figura 5.19: Configuración de la herramienta en Project Settings

La figura 5.20 muestra el diagrama de clases de las opciones de configuración

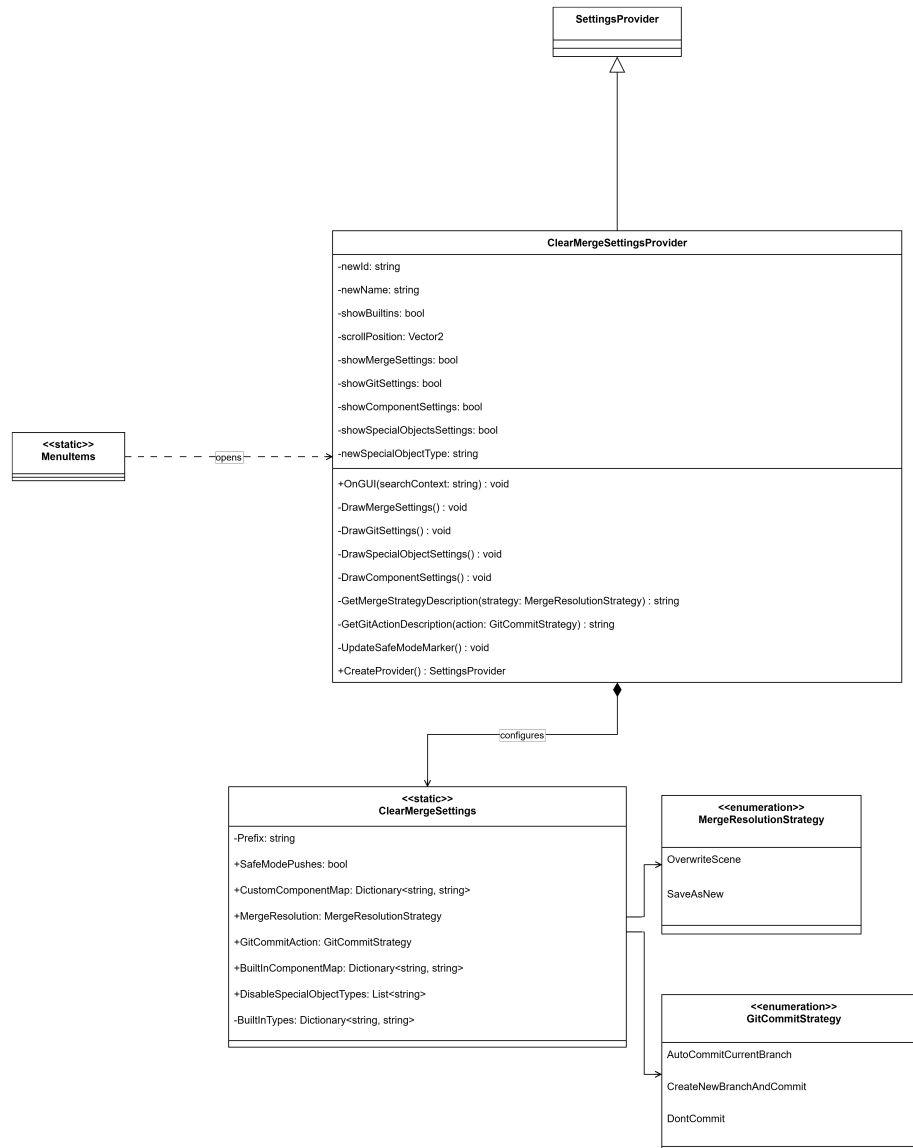


Figura 5.20: Diagrama de clases del sistema de configuración

5.5.3 Implementación

Las nuevas clases implementadas son las siguientes:

GitManager

Clase estática que gestiona la integración con **Git** para realizar operaciones básicas como commit y push, así como obtener información de ramas y archivos.

- `CommitAndPushResolvedScene()`: Ejecuta `git add`, `commit` y `push` so-

bre la **escena** generada tras la resolución de **conflictos**.

- `PushChanges()`: Lanza un `git push` directo al repositorio y muestra el resultado en la consola.
- `GetFileContent(filePath, revision)`: Recupera el contenido de un archivo en una revisión específica (por defecto HEAD).

ClearMergeSettingsProvider

Clase encargada de construir la interfaz de configuración de la herramienta dentro del panel de preferencias de **Unity**.

- `OnGUI()`: Dibuja la interfaz completa del panel de configuración, incluyendo secciones colapsables y campos de entrada.
- `CreateProvider()`: Método anotado con `[SettingsProvider]` que registra este panel en **Unity** bajo `Project > ClearMerge`.

ConflictViewerWindow

Se añadieron las funciones para previsualizar y guardar la **escena** final generada:

- `CreatePreviewScene()`: Crea una **escena** temporal combinando los **GameObjects** elegidos para su inspección previa.
- `SaveFinalScene()`: Guarda la versión final unificada en disco como una nueva **escena** de **Unity**.

5.5.4 Pruebas

Durante este sprint se verificó que el sistema fuera capaz de generar correctamente una **escena** final consolidada a partir de los **GameObjects** seleccionados desde las versiones **BEFORE**, **AFTER** o una combinación de ambas. Esta **escena** fue comprobada para asegurar que fuera funcional, editable y coherente en su estructura interna.

También se confirmó que el archivo resultante se guardaba adecuadamente en disco, siguiendo la estrategia elegida por el usuario (sobrescritura o generación con sufijo). Se evaluó el comportamiento del sistema de commits automáticos, tanto en la rama activa como al crear una rama nueva, validando que las operaciones `add`, `commit` y `push` funcionaran sin errores.

Asimismo, se comprobó que el **modo seguro** bloqueaba correctamente los intentos de hacer push si se detectaban conflictos sin resolver, mediante la presencia de marcadores conflictivos en archivos **YAML**. Por último, se aseguró que las preferencias del usuario configuradas en el panel se mantuvieran de forma persistente entre ejecuciones distintas del editor. El Sprint Review permitió validar la integración automática con **Git** y la persistencia de configuraciones, actualizando el Product Backlog según los resultados obtenidos.

5.6 Sprint 6: Extensión a Assets

Antes de este sprint, se revisó el *Product Backlog* para priorizar la resolución de **conflictos** en *assets* y archivos *.meta*. Este sprint amplió la herramienta para abarcar materiales, **Prefabs**, scripts, texturas y archivos *.meta*, cubriendo así más casos críticos en el desarrollo colaborativo.

5.6.1 Análisis

Se desarrollaron funcionalidades correspondientes a las siguientes historias de usuario:

- **HU-013:** Detección y resolución manual de **conflictos** en *assets*.
- **HU-014:** Resolución automática de **conflictos** en archivos *.meta*.

Los casos de uso derivados se resumen en la Figura 5.21:



Figura 5.21: Casos de uso del sistema de resolución de **conflictos** en *assets*

CU-08: Detectar conflictos en Assets

- **Descripción:** El sistema escanea de forma recursiva el directorio `/Assets` del proyecto, buscando archivos con marcas de **conflicto** generadas por **Git**. Esto incluye tanto archivos de texto (**YAML**) como binarios.
- **Precondiciones:** Existen **conflictos** activos en materiales, **Prefabs**, animaciones u otros archivos del proyecto.

CU-09: Visualizar conflictos en Assets

- **Descripción:** El usuario puede explorar y comparar las versiones en **conflicto** de un asset, ya sea visualizando diferencias entre propiedades **YAML** o previsualizando archivos binarios antes y después del merge.
- **Precondiciones:** Deben haberse detectado previamente **conflictos**.

CU-10: Resolver **conflictos por Asset**

- **Descripción:** A través de la interfaz, el usuario decide qué versión mantener para cada sección o propiedad de un asset. En el caso de binarios, se elige entre la versión completa **BEFORE** o **AFTER**.
- **Precondiciones:** El asset está siendo visualizado.

CU-11: Resolver **conflictos en Meta**

- **Descripción:** Se ejecuta una rutina automática que detecta **conflictos** en archivos `.meta` (principalmente GUID duplicados) y los resuelve sin intervención del usuario.
- **Precondiciones:** Existen archivos `.meta` en **conflicto** en el proyecto.

5.6.2 Diseño

El sistema de resolución de **conflictos** de assets replica en gran medida la estructura desarrollada previamente para **escenas**. Se optó por copiar y adaptar su arquitectura a un nuevo módulo independiente, `ClearMerge.Assets`, con el objetivo de evitar dependencias cruzadas y mantener flexibilidad para futuras integraciones con herramientas externas.

El sistema opera sobre tres tipos de **conflictos**:

- **Conflictos en archivos de assets:** como `.prefab`, `.mat`, `.controller`, donde se detectan y resuelven diferencias entre versiones mediante comparación de propiedades, ya que su información se guarda en formato **YAML**.

- **Conflictos en archivos binarios:** como .png, .fbx, .mp3, que no pueden analizarse a nivel de propiedades. El sistema detecta estos **conflictos** y permite al usuario previsualizar externamente ambas versiones del archivo y seleccionar cuál conservar como resolución.
- **Conflictos en archivos .meta:** que se resuelven automáticamente mediante la generación o sustitución de GUIDs.

Estructura general del sistema

El núcleo del sistema de resolución de **conflictos** en assets se apoya en un conjunto de clases similares al módulo de **escenas**, encargadas de analizar, visualizar y resolver los **conflictos** en Assets.

El proceso comienza analizando recursivamente el proyecto en busca de archivos con marcadores de **conflictos**, carpeta por carpeta partiendo de la raíz /Assets.

Los **conflictos** detectados se encapsulan usando estructuras siguiendo el Patron DTO. En particular:

- `AssetConflict` representa un asset con **conflicto**.
- Cada **conflicto** se estructura en una o más instancias de `PropertyConflict`, clase común con el módulo Scenes, que almacena el contenido de la propiedad **BEFORE** y **AFTER**

Se ha creado otra ventana dentro del editor para visualizar los **conflictos** que permite al usuario visualizar y resolver los **conflictos** de forma interactiva similar a las **escenas**, implementada en `ConflictViewerWindow`:

- Muestra una lista con todos los archivos en **conflicto**.
- Permite inspeccionar propiedades lado a lado y elegir cuál versión conservar.
- Ofrece acciones rápidas como “Seleccionar todo”..

Una vez que el usuario toma sus decisiones de resolución, la clase `ConflictResolver` aplica estos cambios sobre el contenido del archivo. Una vez aplicados y guardados los cambios, se incorpora automáticamente el archivo al índice de **Git**, dejándolo preparado para el siguiente commit.

En paralelo, para manejar **conflictos** en archivos `.meta`, se diseñó una solución completamente automática mediante la clase `MetaConflictResolver`. Este tipo de **conflictos** se producen cuando dos desarrolladores generan el mismo archivo con GUIDs distintos. El sistema:

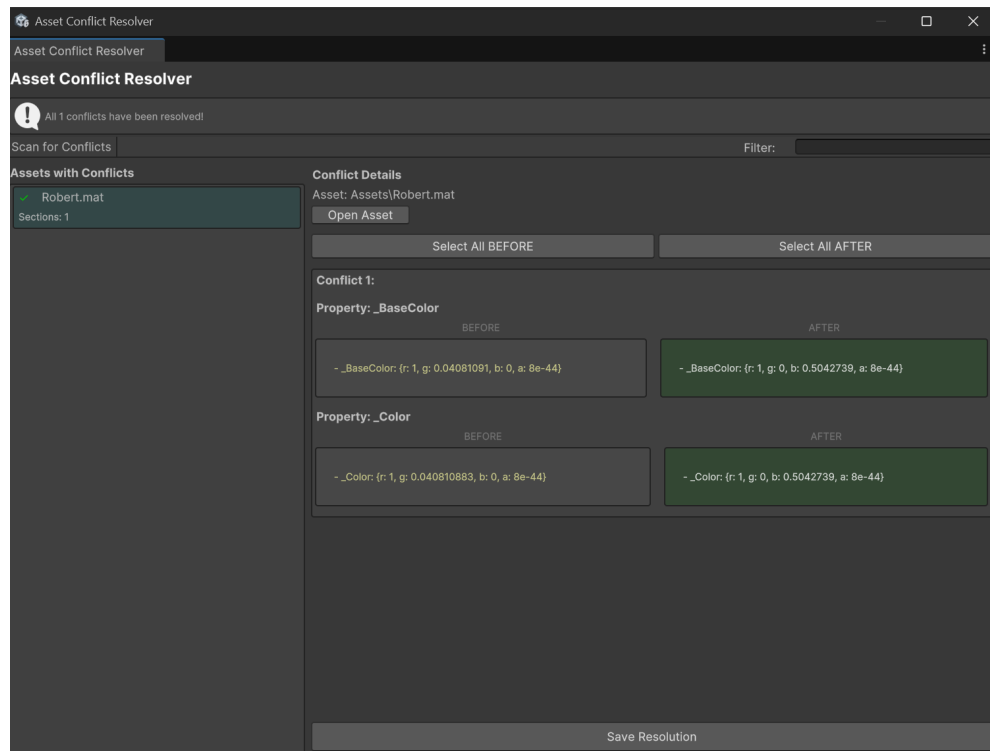
- Detecta el **conflicto** de GUIDs.
- Escoge el GUID más reciente o genera uno nuevo si es necesario.
- Reescribe el archivo `.meta` sin requerir intervención manual.

Además, para facilitar el acceso a estas herramientas desde la interfaz de **Unity**, se añadieron nuevas entradas al menú `ClearMerge > Assets`, desde donde es posible abrir directamente la ventana de resolución de **conflictos** de assets o ejecutar un escaneo automático de **conflictos** en archivos `.meta`.

Interfaz de usuario

La Figura 5.22 muestra la interfaz de resolución de **conflictos** diseñada para assets. Esta interfaz está construida sobre la clase `ConflictViewerWindow`.

- En la parte izquierda se presenta una lista de assets con **conflictos**.
- Al seleccionar uno, se muestra a la derecha una comparación lado a lado de las propiedades modificadas (**BEFORE** y **AFTER**).
- El usuario puede hacer clic sobre cada propiedad para elegir una versión, o utilizar los botones `Select All BEFORE` y `Select All AFTER`.
- También se ofrece un botón *Open Asset* para abrir directamente el archivo y ubicarlo en el proyecto.
- Una vez resueltas todas las diferencias, el botón *Save Resolution* se activa, y el contador de **conflictos** no resueltos se actualiza dinámicamente.

Figura 5.22: Interfaz de usuario para resolución de **conflictos** en assets

En caso de ser los archivos conflictivos binarios, aparecerían dos botones en la lista de propiedades para abrir externamente las dos versiones de archivo.

Diagrama de clases del sistema

El diagrama de clases de la Figura 5.23 resume la arquitectura del módulo de assets, mostrando las relaciones entre los componentes clave: el analizador, la vista, la lógica de resolución, el sistema automático de archivos `.meta` y las estructuras de datos auxiliares.

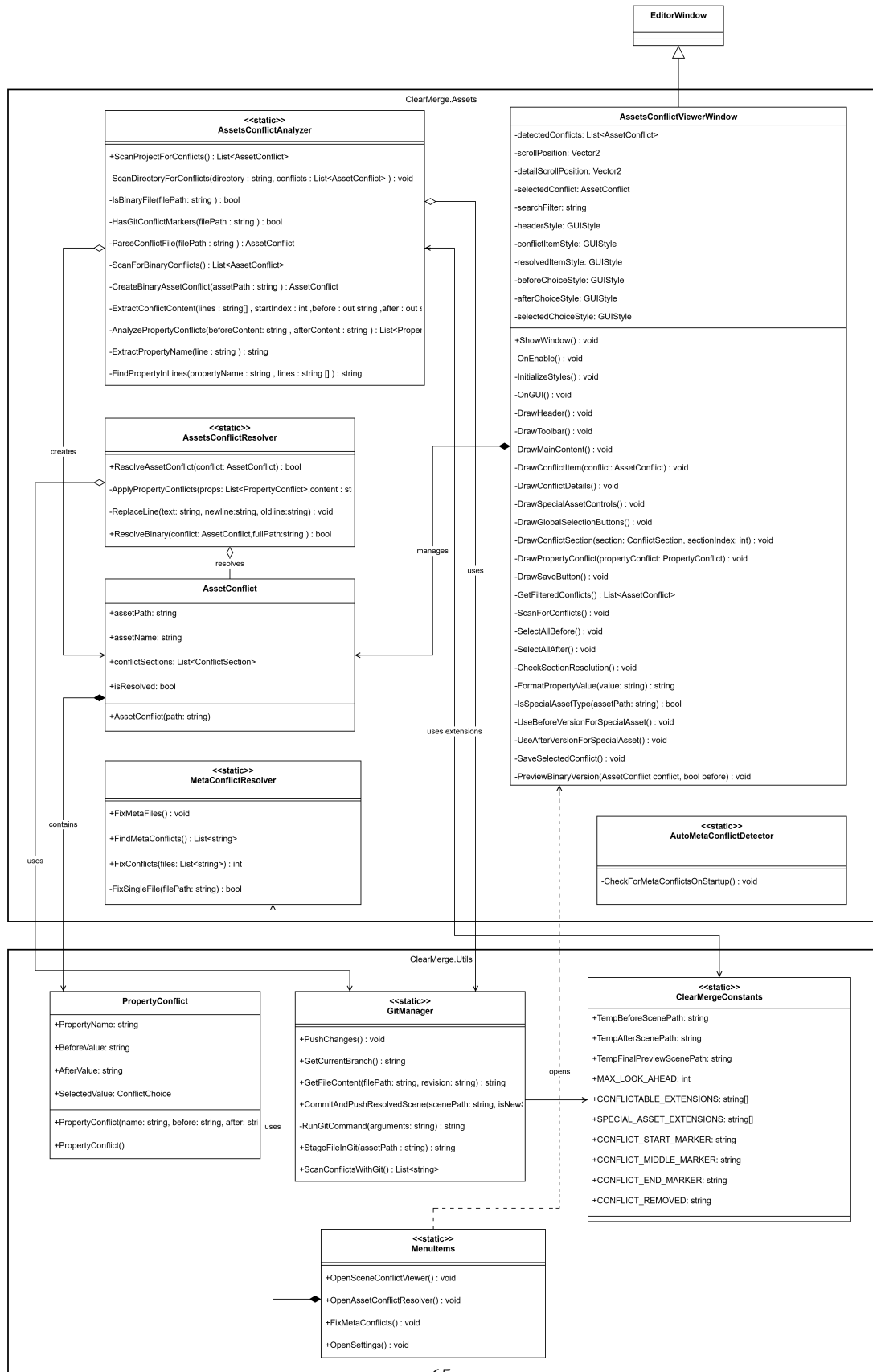


Figura 5.23: Diagrama de clases del sistema de resolución de conflictos en assets

5.6.3 Implementación

Las nuevas clases implementadas para extender el sistema a los assets de [Unity](#) son las siguientes:

ConflictAnalyzer

Clase encargada de escanear todos los archivos del proyecto para detectar [conflictos](#) en assets editables (como `.prefab`, `.mat`, `.cs`, etc.).

- `ScanProjectForConflicts()`: Función principal que recorre todo el proyecto Unity para detectar conflictos, tanto en archivos de texto como binarios. Retorna una lista de `AssetConflict`.
- `ParseConflictFile()`: Parsea un archivo con marcas de conflicto de Git. Extrae su contenido y genera un objeto `AssetConflict` con la información necesaria para su resolución.
- `AnalyzePropertyConflicts()`: Compara dos versiones de un mismo archivo (antes y después del conflicto) y detecta diferencias a nivel de propiedades serializadas (como campos de componentes en YAML).
- `ScanForBinaryConflicts()`: Detecta conflictos en archivos binarios (como imágenes, prefabs o materiales) utilizando el estado de Git, y los convierte en conflictos tratables por la herramienta.

ConflictResolver

Clase encargada de aplicar las resoluciones elegidas por el usuario sobre los archivos conflictivos y generar una nueva versión final.

- `ResolveAssetConflict()`: Función principal del módulo de resolución. Aplica los cambios seleccionados por el usuario para resolver un [conflicto](#), ya sea en un archivo de texto o binario.
- `ResolveBinary()`: Gestiona la resolución de conflictos en archivos binarios. Copia la versión seleccionada (antes o después) al archivo final según la elección en las propiedades.
- `ApplyPropertyConflicts()`: Aplica los cambios seleccionados a nivel de propiedad. Reemplaza líneas específicas en el contenido del archivo y elimina marcas de [conflicto](#).

ConflictViewerWindow

Ventana de **Unity** especializada en mostrar y permitir la resolución visual de **conflictos** en assets.

- `OnGUI()`: Dibuja la interfaz gráfica principal, incluyendo encabezado, barra de herramientas, lista de conflictos y detalles del conflicto seleccionado.
- `DrawConflictDetails()`: Muestra los detalles del **conflicto** seleccionado, permitiendo elegir entre versiones y propiedades.

MetaConflictResolver

Módulo específico para la detección y resolución automática de **conflictos** en archivos `.meta`.

- `FindMetaConflicts()`: Recorre todos los archivos `.meta` del proyecto y detecta aquellos que presentan **conflictos** activos.
- `FixMetaFiles()`: Llama internamente a `FindMetaConflicts()` y ejecuta la resolución automática sobre todos los archivos conflictivos encontrados. Este método se lanza directamente al invocar la opción del menú `ClearMerge > Assets > Scan Meta Conflicts`.

Este proceso es completamente automático y no requiere intervención del usuario, lo que lo convierte en una herramienta clave para mantener la integridad del proyecto durante fusiones complejas.

Gestión de conflictos en archivos binarios

Los **conflictos** en archivos binarios (*texturas, modelos, animaciones, etc.*) no pueden resolverse línea a línea como los archivos **YAML**. Por ello, se implementó un tratamiento especial para estos casos.

El sistema detecta **conflictos** binarios utilizando el comando `git status --porcelain`, filtrando archivos con estados AA (both added) y UU (both modified). Si el archivo tiene una extensión registrada como binaria (según `ClearMergeConstants.BINARY_ASSET_EXTENSIONS`), se crea una sección de **conflicto** especial en la clase `AssetConflict`, indicando que el usuario debe elegir entre una versión completa.

Cambios en clases

A la clase `GitManager` se le añadieron dos nuevas funciones para añadir archivos al commit (`StageFileInGit()`) y también hacer el escaner de **conflictos** en el proyecto `ScanConflictsWithGit()`.

Así también se decidió mover la clase `PropertyConflict` a el módulo `Utils`, ya que es una clase común entre ambos módulos `Scenes` y `Assets`, y puede ser reutilizada

5.6.4 Pruebas

Las funcionalidades fueron validadas mediante los siguientes escenarios: En primer lugar, se probaron archivos con **conflictos** de contenido en formato **YAML**, como materiales (`.mat`) y **Prefabs**. Estos archivos contienen múltiples propiedades serializadas, susceptibles a modificaciones por distintos usuarios. La herramienta permite al usuario comparar visualmente cada propiedad y elegir entre las versiones **BEFORE** o **AFTER**, facilitando una resolución precisa y controlada de los cambios.

También se consideró el caso de archivos `.meta` duplicados que comparten el mismo **GUID**. Esta situación es particularmente crítica, ya que **Unity** utiliza estos identificadores de forma interna para enlazar referencias entre **assets**. La herramienta detecta automáticamente este tipo de **conflicto** y alerta al usuario, previniendo referencias rotas o comportamientos inesperados en el proyecto.

Otro escenario evaluado consistió en **conflictos** entre archivos distintos que comparten el mismo nombre y se ubican en la misma ruta dentro del repositorio. Aunque se trate de versiones diferentes (por contenido o historial de **Git**), **Unity** los interpreta como equivalentes, lo que puede generar sobrescrituras no deseadas. La herramienta fue capaz de identificar correctamente estas diferencias y presentarlas de forma clara, permitiendo al usuario seleccionar conscientemente cuál versión conservar.

Finalmente, se analizaron casos en los que los archivos involucrados eran **archivos binarios** o no legibles directamente (como imágenes, animaciones o ciertos modelos 3D). En estos casos, la herramienta ofrece la posibilidad de previsualizar cada versión externamente, lo que permite al usuario inspeccionar el contenido real antes de tomar una decisión, manteniendo la utilidad incluso en escenarios donde el contenido no puede interpretarse dentro del editor. En el Sprint Review se verificó la funcionalidad para **assets** y se identificaron mejoras para la gestión de archivos binarios y meta.

5.7 Sprint 7: Integración y Documentación

Previo al Sprint 7, se revisó el *Product Backlog* y se priorizó la integración de herramientas y la detección proactiva de **conflictos**. En este sprint se unificaron las herramientas de resolución de **escenas** y assets en una sola ventana, se automatizó por completo el proceso de instalación y se desarrolló un sistema que alerta al desarrollador cada vez que se refresca el editor.

5.7.1 Análisis

Se desarrollaron funcionalidades correspondientes a las siguientes historias de usuario:

- **HU-004:** Como desarrollador **Unity**, quiero que el sistema detecte automáticamente **conflictos** en mis **escenas** para no tener que revisar manualmente cada archivo después de un merge.
- **HU-015:** Como desarrollador, quiero ser alertado inmediatamente cuando existan **conflictos** en mi proyecto al abrir **Unity**.
- **HU-016:** Como desarrollador, quiero una configuración automática inicial que prepare mi proyecto sin configuración manual.
- **HU-017:** Como desarrollador, quiero acceder a todas las herramientas de resolución de **conflictos** desde un punto central unificado.
- **HU-018:** Como desarrollador, quiero ver estadísticas en tiempo real del estado de **conflictos** en mi proyecto.

Los casos de uso derivados se resumen en la Figura 5.24. Con esto se concluye el diagrama final de casos de uso, se puede ver con mayor detalle en la tabla del apéndice A.2:

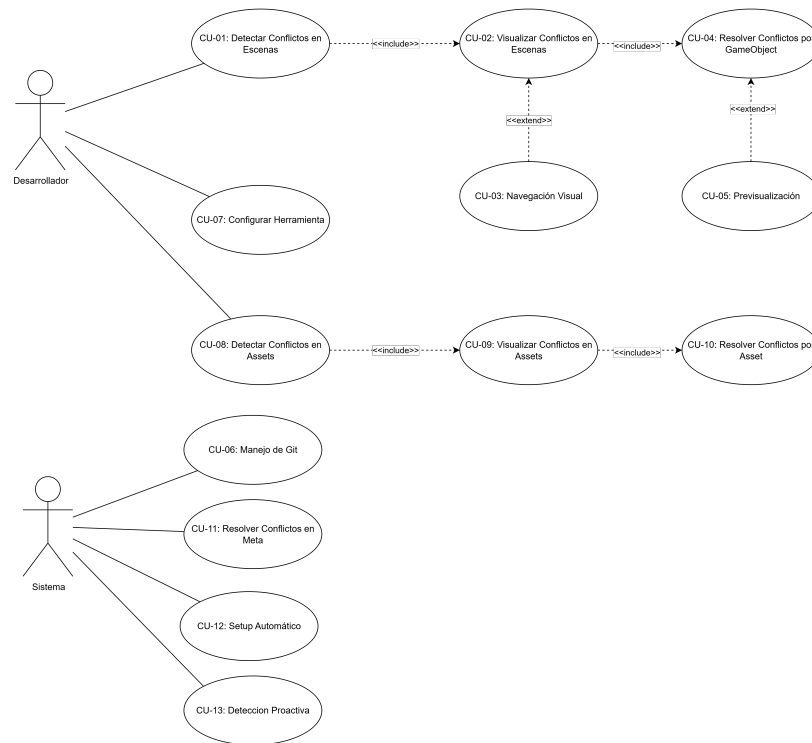


Figura 5.24: Casos de uso unificados para el Sprint 7

CU-12: Setup Automático

- **Descripción:** El sistema configura el entorno automáticamente al iniciar la herramienta.

CU-13: Detección Proactiva

- **Descripción:** El sistema detecta automáticamente **conflictos** en todo el entorno de trabajo.

5.7.2 Diseño

En este sprint se unificaron las herramientas de resolución de **conflictos** de **escenas** y **assets** en una única interfaz de usuario, accesible desde el menú principal de **Tools/ClearMerge**. Al abrir el proyecto o al refrescar los **assets**, el sistema realiza automáticamente un escaneo completo en busca de cualquier **conflicto** (incluyendo aquellos en archivos binarios que **Unity** no notifica por defecto). Cuando se detectan discrepancias, aparece inmediatamente una ventana emergente que informa al usuario de la existencia y el número de **conflictos** encontrados, y ofrece un botón para abrir la nueva ventana principal (Figura 5.25).

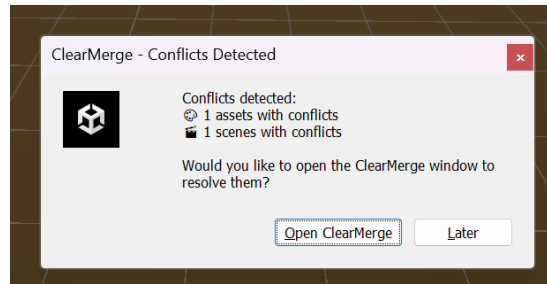


Figura 5.25: Ventana emergente de alerta de **conflictos** al iniciar o refrescar el proyecto

La ventana principal presenta un panel de resumen donde se muestra el conteo de **conflictos** de **escenas**, assets y archivos **.meta**, junto con accesos directos a cada uno de los visores de **conflictos** y a la configuración de la herramienta (Figura 5.26). Además, se añadió un atajo en la barra de menús para forzar manualmente el escaneo de **conflictos** en cualquier momento.

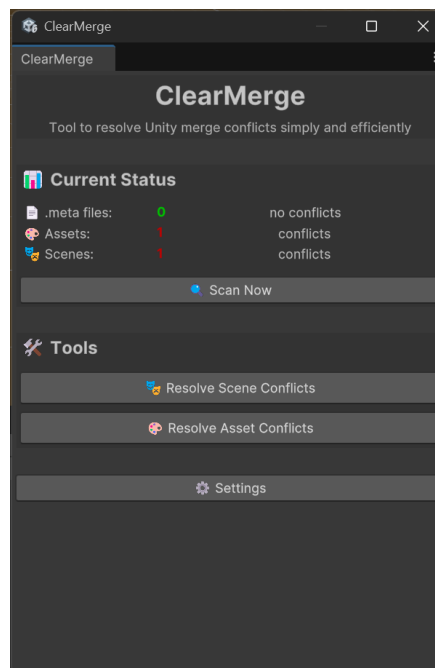


Figura 5.26: Interfaz principal unificada con accesos directos

Para agilizar la puesta en marcha, se implementó un *Setup Automático* que se lanza la primera vez que se abre cualquier ventana de la herramienta (o desde el menú), creando las carpetas temporales necesarias, generando archivos de configuración de **Git** (por ejemplo, reglas en **.gitignore**) y ajustando parámetros iniciales de **Unity** (Figura 5.27). Este asistente garantiza que el entorno está correctamente preparado, aunque se requiere la instalación previa, especificada en el apéndice B.2.

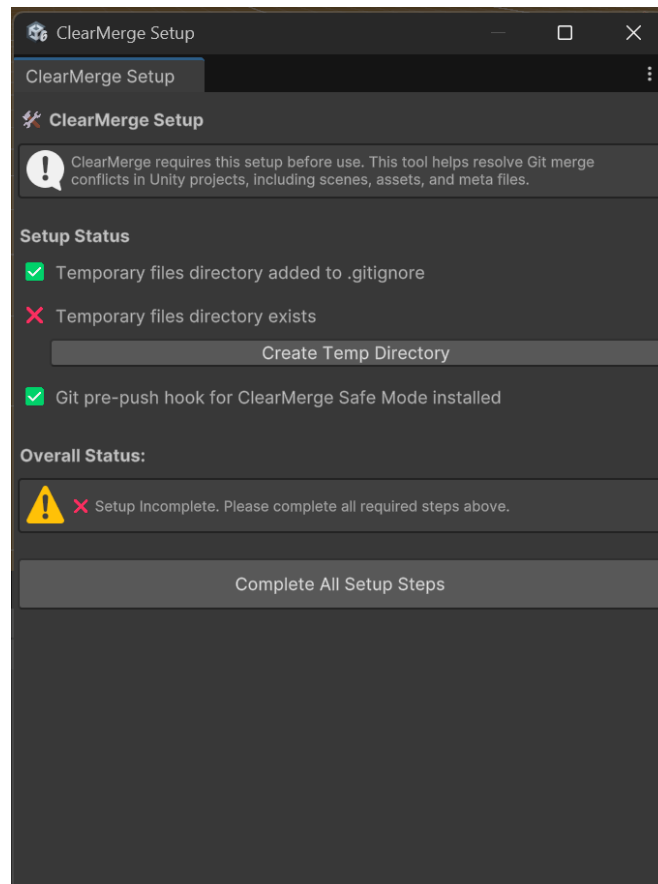


Figura 5.27: Asistente de configuración automática al instalar la herramienta

A nivel de código, se extendió el módulo `ClearMerge.Utils` con nuevas clases para soportar la detección proactiva y el setup inicial. El diagrama de clases (Figura 5.28) muestra cómo `GlobalConflictDetector` orquesta los escaneos automáticos, y cómo `SetupManager` crea y verifica la estructura de carpetas y archivos.

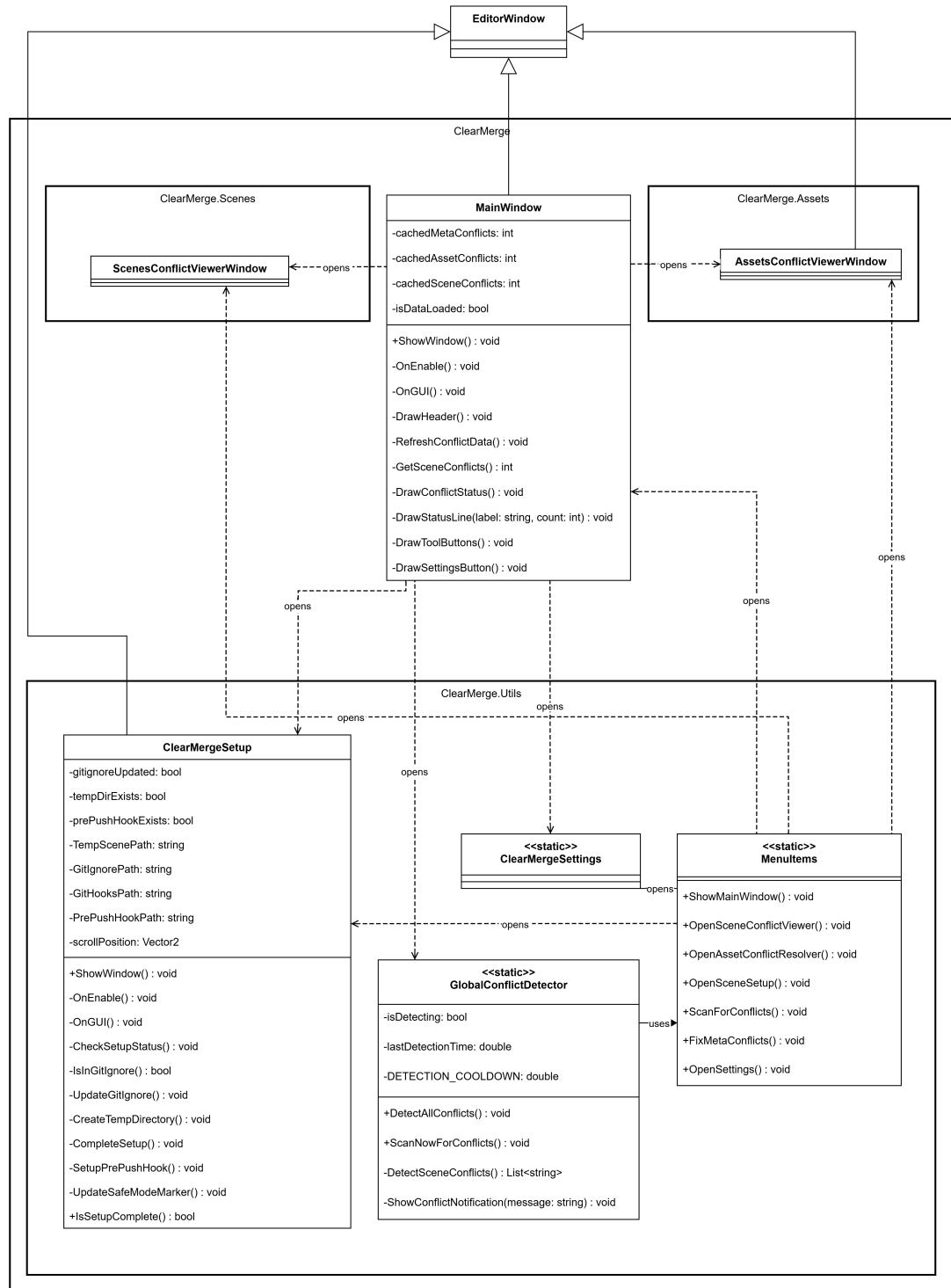


Figura 5.28: Diagrama de clases de los componentes de configuración y detección automática

Por último, se enriqueció el análisis de **conflictos** en **escenas** para cubrir escenarios complejos que antes quedaban fuera de alcance:

Conflictos cruzados Los *conflictos cruzados* surgen cuando la modificación de una referencia (por ejemplo, cambiar el padre de un **GameObject**) impacta indirectamente en otros **GameObjects** de la **escena**. Para ello se incorporó la clase **CrossReferenceConflict**, asociada a cada **ComponentConflict** (Vase Fig 5.29), que registra el tipo de referencia afectada (como cambio de padre o dependencia) y los identificadores antes y después de la fusión. Gracias a esto, al resolver **conflictos** de jerarquía, todas las referencias relacionadas se detectan y muestran al usuario para que pueda resolverlas de forma coherente.

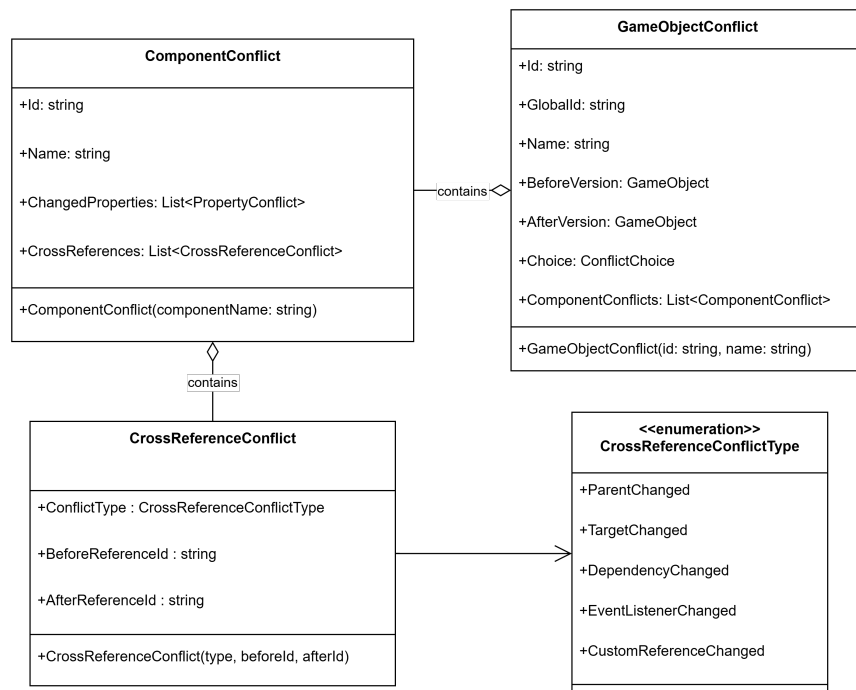


Figura 5.29: Diagrama de clases mostrando la incorporación de **CrossReferenceConflict** en el módulo de **escenas**

GameObjects eliminados vs. modificados En muchos merges reales, un desarrollador puede eliminar por completo un **GameObject** mientras otro realiza cambios en sus propiedades. El sistema ahora distingue estos dos escenarios y presenta al usuario la opción de conservar la versión donde el **GameObject** existe (con todas sus modificaciones) o la versión donde ha sido eliminado, garantizando que la jerarquía resultante no quede en un estado inconsistente.

Bloques duplicados en YAML En ocasiones, [Unity](#) puede generar regiones de [YAML](#) duplicadas para un mismo [GameObject](#), lo que provoca identificadores repetidos y rompe el mapeo durante la resolución. Para evitarlo, al finalizar el análisis se eliminan automáticamente las regiones repetidas, conservando únicamente la definición original de cada [GameObject](#). De este modo, la [escena](#) final queda libre de entradas duplicadas y lista para cargarse sin errores de IDs reiterados.

Estos tres mecanismos, detección de referencias cruzadas, distinción entre eliminación y modificación de [GameObjects](#), y limpieza de bloques duplicados, amplían la robustez de la herramienta y mejoran la fiabilidad del proceso de merge en entornos colaborativos de [Unity](#).

5.7.3 Implementación

GlobalConflictDetector

Clase responsable de orquestar la detección unificada de todos los tipos de [conflictos](#) al inicio o refresco del proyecto.

- `ScanAll()`: Escanea de forma recursiva tanto los assets como las [escenas](#) en busca de [conflictos](#) (marcadores [Git](#), binarios y meta-files).
- `OnProjectLoaded()`: Invocado automáticamente al abrir o refrescar [Unity](#); dispara `ScanAll()` y, si se detectan [conflictos](#), muestra un diálogo de alerta al usuario.

SetupManager

Encargado de preparar el entorno de la herramienta la primera vez que se utiliza la herramienta, garantizando la configuración necesaria en el proyecto.

- `EnsureFolders()`: Crea las carpetas temporales
- `WriteGitIgnore()`: Genera o actualiza las reglas en `.gitignore` para incluir los archivos temporales de la herramienta.
- `MarkInitialized()`: Almacena en `EditorPrefs` un flag que indica que el setup se completó correctamente, evitando repetirlo.

CrossReferenceConflict

Nueva clase para representar [conflictos](#) en las referencias entre [GameObjects](#) dentro de una [escena](#) (p. ej. jerarquía padre/hijo, dependencias de [componentes](#), listeners de eventos).

- `ConflictType`: enumera el tipo de referencia que cambió, aunque solo se implementara `ParentChanged` en este desarrollo.

- `BeforeReferenceId`, `AfterReferenceId`: identificadores del objeto referenciado antes y después de la fusión.

Estos `CrossReferenceConflict` se incluyen en la lista `CrossReferences` de cada `ComponentConflict`, de modo que al resolver un `componente` se actualicen también todas sus relaciones cruzadas.

ConflictAnalyzer (Escenas)

Amplía el análisis de `escenas` incorporando la detección de `conflictos` cruzados y la limpieza de bloques duplicados.

- `AnalyzeConflictCrossReferences()`:
 - Recorre las propiedades en `conflicto` de cada `componente` detectado.
 - Detecta cambios en referencias (p. ej. padre/hijo, target de un `componente`, listas de dependencias o listeners).
 - Genera instancias de `CrossReferenceConflict` con los IDs **BEFORE/AFTER**.
- `RemoveDuplicateBlocks()`:
 - Tras reunir todas las resoluciones, filtra y elimina automáticamente bloques `YAML` repetidos en la `escena`.
 - Garantiza que no queden `fileIDs` duplicados y que la `escena` resultante cargue sin errores.

5.7.4 Pruebas

Las funcionalidades se validaron con distintos escenarios típicos de trabajo en `Unity` con `Git`. Se probaron `escenas` con bloques `YAML` duplicados, que causan errores por `fileID` repetidos; la herramienta los detectó y eliminó correctamente.

Se evaluaron también cambios conflictivos de jerarquía entre `GameObjects`, donde uno era movido de lugar en una rama y modificado en otra. Gracias al análisis de referencias cruzadas, se resolvieron con contexto y sin pérdida de información.

Otro caso fue el de `GameObjects` eliminados en una rama y editados en otra. El sistema mostró ambas opciones y permitió al usuario decidir con claridad.

Por último, se validaron conflictos en `archivos binarios` y archivos `.meta` con `GUIDs` duplicados. En ambos casos, la detección fue automática y se ofrecieron medios adecuados para resolverlos o visualizarlos externamente.

Conclusiones

ESTE proyecto tuvo como objetivo principal el desarrollo de una herramienta que simplifique la resolución de conflictos en proyectos colaborativos de [Unity](#). Tras completar un proceso de desarrollo iterativo e incremental que abarcó desde la comprensión inicial del problema hasta la integración final de un sistema completo, se pueden extraer conclusiones importantes sobre el trabajo realizado y los resultados obtenidos.

6.1 Objetivos cumplidos

Se han conseguido cumplir todos los objetivos planteados. Se desarrolló una herramienta funcional que permite resolver y visualizar de forma clara los conflictos tanto en [escenas](#) como en assets, dejando atrás la tarea tediosa y compleja de interpretar manualmente los archivos [YAML](#).

Además, se logró automatizar parte del proceso, permitiendo detectar conflictos de forma automática y facilitar su subida nuevamente al repositorio en la nube, agilizando así el flujo de trabajo en equipos colaborativos.

Aunque la herramienta es relativamente simple en cuanto a su alcance, cumple con éxito lo que se propuso: asistir en la resolución de conflictos dentro de proyectos [Unity](#), algo que hasta ahora resultaba incómodo y poco intuitivo.

El simple hecho de poder visualizar de forma clara una [escena](#) conflictiva, algo que [Unity](#) no permite por defecto de manera efectiva, o la posibilidad de regresar la [escena](#) a un estado anterior, representa un avance considerable y una funcionalidad que cualquier desarrollador agradecería tener en su flujo de trabajo diario.

6.2 Aprendizajes y aportes personales

Este proyecto no solo ha sido una experiencia técnica, sino también una oportunidad para aplicar y reforzar muchos conocimientos adquiridos durante la carrera.

Por un lado, he aprendido a trabajar con mayor profundidad en el editor de [Unity](#). Aunque ya estaba familiarizado con su uso básico, nunca antes me había enfrentado a la tarea de modificar su interfaz ni de desarrollar herramientas internas personalizadas. Esto me permitió también explorar recursos compartidos por la comunidad de [Unity](#), que demuestra una fuerte orientación a facilitar la vida de los desarrolladores mediante extensiones y utilidades.

También mejoré notablemente mi manejo de $\text{\LaTeX}$, tanto en estructura como en automatización, lo cual me ha resultado útil no solo para este documento, sino que será de gran ayuda en cualquier contexto académico o técnico en el futuro.

En cuanto al propio funcionamiento de [Unity](#), comprendí mejor cómo está estructurado internamente el motor y cómo se gestionan realmente sus recursos y archivos. Este conocimiento interno del sistema me ha ayudado a entender por qué surgen los conflictos y cómo pueden abordarse desde el código.

Por último, este proyecto me permitió ver en la práctica cómo muchas de las herramientas y metodologías que aprendimos en clase, como los patrones de diseño, la programación modular o la integración continua, tienen una aplicación real y tangible en proyectos de desarrollo. Más allá de ser teoría, se traducen en decisiones que facilitan el trabajo, previenen errores y permiten alcanzar resultados útiles y sostenibles.

6.3 Proyección futura

Este proyecto me ha ayudado a profundizar aún más en el mundo del desarrollo de videojuegos, que siempre ha sido un ámbito de especial interés personal, y una de las razones por las que en aquel entonces me intereso esta carrera. Me hizo especial ilusión poder desarrollar una herramienta que pueda ser útil para otras personas, tanto a mis compañeros de carrera como a otros desarrolladores. Hace algunos años, yo mismo necesitaba soluciones como esta, y ahora tengo la posibilidad de compartir algo que puede facilitar el trabajo de otros. Poder devolver a la comunidad una parte de todo lo que me enseñó sobre este motor es algo que valoro mucho y que me anima a seguir colaborando con ella.

Creo que la experiencia obtenida aquí puede ser una buena base para futuras oportunidades profesionales relacionadas con la creación de videojuegos, ya sea en estudios o proyectos personales.

Futuros Desarrollos

Las posibilidades de evolución futura de esta herramienta se extienden en múltiples direcciones, desde mejoras incrementales hasta transformaciones paradigmáticas que podrían redefinir metodológicamente el abordaje de conflictos en entornos de desarrollo colaborativo.

7.1 Optimización y Mejoras Técnicas

Existen varios aspectos que reúnen el rendimiento y optimización del proyecto que podrían ser mejorados:

Aspecto	Limitación Actual	Mejora Propuesta
Procesamiento	Secuencial	Async/await paralelo
Memoria	Carga completa	Lazy loading inteligente
Cache	Básico	Sistema predictivo

Tabla 7.1: Hoja de ruta de optimizaciones técnicas

La implementación de **threading mejorado** mediante patrones async/await constituiría una transformación significativa en la experiencia del usuario, particularmente durante el procesamiento de archivos de gran tamaño donde la responsividad del sistema resulta crítica. Un **sistema de caché inteligente** con capacidades predictivas podría anticipar qué archivos requiere examinar el usuario, implementando precarga en segundo plano.

7.2 Otras Mejoras

Aunque la herramienta cumple con los objetivos planteados, siempre hay espacio para mejoras, especialmente en la detección de conflictos más complejos. Cada proyecto y equipo

tiene sus propias convenciones y patrones de trabajo, por lo que resulta casi imposible cubrir todos los casos posibles con una solución única. Una evolución natural sería diseñar un sistema modular de detección de conflictos, que permitiera a los desarrolladores añadir sus propias reglas y extensiones para adaptarse a sus necesidades específicas, agilizando la resolución sin intervención manual.

Por otro lado, las capacidades de personalización de la interfaz de [Unity](#) tienen sus limitaciones y, en ocasiones, pueden introducir inestabilidades. Una alternativa sería desarrollar un editor completamente independiente, construido desde cero y optimizado para gestionar fusiones y conflictos, sin depender exclusivamente de las [API](#) de [Unity](#). Esta aproximación abriría la puerta a una experiencia más robusta y libre de interferencias, garantizando un funcionamiento más consistente incluso en escenarios complejos.

Apéndices

Apéndice A

Material Adicional

ESTE apéndice proporciona información técnica complementaria sobre la herramienta desarrollada, como tablas detalladas de casos de uso o requisitos funcionales de la herramienta.

A.1 Requisitos funcionales de la herramienta

En la tabla A.1 se reflejan los requisitos funcionales del proyecto.

Tabla A.1: Requisitos funcionales de la herramienta

ID	Nombre	Descripción
RF-001	Detección de conflictos Git	Detecta automáticamente conflictos en archivos <code>.scene</code> mediante el análisis de los marcadores estándar de Git .
RF-002	Análisis YAML	Analiza la estructura YAML para identificar GameObjects y componentes en conflicto .
RF-003	Visualización clara	Muestra de forma comprensible las diferencias entre versiones antes y posteriores al merge.

..... (continúa en la siguiente página)

Tabla A.1 – (viene de la página anterior)

ID	Nombre	Descripción
RF-004	Resolución granular	Permite al desarrollador resolver cada <code>GameObject</code> de forma individual.
RF-005	Previsualización de cambios	Genera una vista previa visual de los cambios antes de confirmarlos.
RF-006	Integración con <code>Git</code>	Automatiza commits tras la resolución.
RF-007	Soporte para otros assets	Extiende la funcionalidad a <code>conflictos</code> en assets.
RF-008	Resolución de <code>.meta</code>	Resuelve automáticamente <code>conflictos</code> en archivos <code>.meta</code> .
RF-009	Detección proactiva	Alerta al desarrollador sobre <code>conflictos</code> existentes al iniciar <code>Unity</code> .
RF-010	Configuración personalizable	Permitir configurar la herramienta a gusto del desarrollador.
RF-011	Prevención de errores	Implementa medidas de seguridad para evitar la reintroducción de marcadores de <code>conflicto</code> en el repositorio.

A.2 Casos de uso de la herramienta

Tras el desarrollo de la herramienta estos fueron los casos de uso descritos en las tablas A.2 y A.3:

Tabla A.2: Casos de uso identificados para la herramienta

ID	Nombre	Descripción
CU-01	Detectar Conflictos en Escenas	Escanea archivos ‘scene’ y detecta automáticamente marcadores de conflicto presentes en el código generado por Unity.
CU-02	Visualizar Conflictos en Escenas	Permite comparar visualmente versiones HEAD e incoming de objetos en conflicto dentro de la escena.
CU-03	Navegación Visual	Herramientas de zoom, enfoque y alternancia visual durante la resolución de conflictos.
CU-04	Resolver Conflictos por GameObject	Permite elegir la versión deseada de cada objeto para resolver conflictos en una escena.
CU-05	Previsualización	Muestra el resultado final de los objetos seleccionados antes de aplicar la resolución.
CU-06	Manejo de Git	Ejecuta automáticamente operaciones como commit y push después de resolver conflictos.
CU-07	Configurar Herramienta	Define opciones de visualización, preferencias y parámetros de resolución.
CU-08	Detectar Conflictos en Assets	Detecta conflictos en assets como materiales, prefabs y otros elementos del proyecto.
CU-09	Visualizar Conflictos en Assets	Compara visualmente versiones de assets conflictivos.

..... (continúa en la siguiente página)

Tabla A.2 – (viene de la página anterior)

ID	Nombre	Descripción
CU-10	Resolver Conflictos por Asset	Permite resolver conflictos seleccionando versiones para cada asset.
CU-11	Resolver Conflictos en Meta	Resuelve automáticamente los conflictos presentes en archivos ‘meta’ generando nuevos GUID.
CU-12	Setup Automático	Configura el entorno automáticamente, incluyendo carpetas y archivos de configuración.
CU-13	Detección Proactiva	Escanea continuamente para detectar y notificar conflictos en todo el entorno de trabajo.

Tabla A.3: Condiciones y flujo de los casos de uso

ID	Precondiciones	Postcondiciones	Flujo Principal
CU-01	Existe un conflicto Git en archivos ‘scene’	Se muestra información detallada de conflictos	1. Escanear archivos ‘scene’ 2. Detectar marcadores de conflicto 3. Analizar estructura YAML 4. Presentar objetos conflictivos
CU-02	Se han detectado conflictos en la escena	El desarrollador visualiza claramente las diferencias	1. Mostrar ventana de resolución 2. Seleccionar objeto 3. Alternar versiones HEAD/incoming

..... (continúa en la siguiente página)

Tabla A.3 – (viene de la página anterior)

ID	Precondiciones	Postcondiciones	Flujo Principal
CU-03	Se está visualizando un conflicto	-	1. Acercar/alejar objetos 2. Alternar vista previa entre versiones
CU-04	Se está visualizando el conflicto	Conflicto resuelto en la escena	1. Elegir versión por objeto 2. Previsualizar resultado 3. Aplicar resolución
CU-05	Se ha seleccionado una versión	-	1. Generar previsualización final 2. Confirmar resolución
CU-06	-	Se ejecutan operaciones Git automáticamente	1. Confirmar resolución final 2. Ejecutar 'commit', 'push'
CU-07	Herramienta instalada	-	1. Abrir menú de configuración 2. Ajustar preferencias de visualización, resolución automática, etc.
CU-08	Conflictos en archivos '.prefab', '.mat', etc.	Se muestra lista de conflictos en assets	1. Escanear proyecto 2. Identificar archivos conflictivos 3. Mostrar lista
CU-09	Detectados conflictos en assets	-	1. Seleccionar asset conflictivo 2. Mostrar diferencias entre versiones
CU-10	Visualización de asset activo	Conflicto resuelto en el asset	1. Elegir versión 2. Aplicar resolución 3. Confirmar resultado
CU-11	Se detectan conflictos en archivos '.meta'	Archivo '.meta' resuelto automáticamente	1. Detectar conflicto 2. Generar nuevo GUID único 3. Resolver automáticamente

..... (continúa en la siguiente página)

Tabla A.3 – (viene de la página anterior)

ID	Precondiciones	Postcondiciones	Flujo Principal
CU-12	-	Herramienta lista para su uso	1. Crear carpetas necesarias 2. Configurar '.gitignore' 3. Preparar entorno Unity
CU-13	-	Notificación mostrada si hay conflictos	1. Escanear entorno en busca de conflictos 2. Notificar al usuario

Instalación y Configuración

ESTE apéndice contiene detalles del proceso de instalación, y configuración recomendada de [Git](#) para proyectos [Unity](#).

B.1 Requisitos del Sistema

Para garantizar el correcto funcionamiento de la herramienta, el entorno debe cumplir los siguientes requisitos mínimos:

- Unity3D versión 6.0 o superior.
- Sistema operativo Windows 10/11 o macOS 10.15 en adelante.
- Git instalado y accesible desde la línea de comandos.
- Memoria RAM mínima de 8 GB (se recomiendan 16 GB).
- Al menos 1 GB de espacio libre en disco.

B.2 Proceso de Instalación

La forma más sencilla y recomendada de instalar la herramienta es mediante un paquete Unity:

1. Descargar el archivo `ClearMerge.unitypackage` desde el repositorio oficial o distribución correspondiente.
2. En el editor de Unity, ir a `Assets > Import Package > Custom Package...`
3. Seleccionar el archivo `ClearMerge.unitypackage`.
4. Marcar todos los elementos en el diálogo de importación y hacer clic en `Import`.

5. Una vez completado, se debe visualizar la opción `Tools > ClearMerge` en la barra superior de menús.
6. Ejecute `Tool > ClearMerge > Utils > Setup` para completar el setup automático de la herramienta.

La herramienta se inicializará automáticamente al abrir cualquiera de sus ventanas.

B.3 Configuraciones de Git Recomendadas

Unity trabaja internamente con archivos en formato YAML, por lo que es importante que Git esté configurado para facilitar una gestión eficiente de merges en estos archivos. A continuación, se presenta una configuración base recomendada para los archivos `.gitignore`:

B.3.1 Archivo `.gitignore`

```
1 /[Ll]ibrary/
2 /[Tt]emp/
3 /[Oo]bj/
4 /[Bb]uild/
5 /[Bb]uilds/
6 /[Ll]ogs/
7 /[Uu]ser[Ss]ettings/
8
9 *.tmp
10 *.user
11 *.userprefs
12 *.pidb
13 *.booproj
14 *.svd
15 *.pdb
16 *.mdb
17 *.opendb
18 *.VC.db
```

Listing B.1: Exclusión de archivos y carpetas generados por Unity

Estas configuraciones ayudan a mantener limpio el repositorio, evitando archivos temporales o específicos del entorno de desarrollo, y asegurando una correcta fusión automática de los archivos YAML manipulados por Unity.

Lista de acrónimos

API Interfaz de Programación de Aplicaciones. [7](#), [80](#)

Git LFS Git Large File Storage. [5](#)

SWOT Strengths, Weaknesses, Opportunities, and Threats. [vi](#), [3](#), [17](#), [18](#)

UML Unified Modeling Language. [19](#)

VCS Version Control System (Sistema de Control de Versiones). [5](#)

Glosario

AFTER Versión entrante durante el *merge*, correspondiente a INCOMING en Git. Contiene los nuevos cambios propuestos. [iv](#), [38](#), [40](#), [45](#), [48–52](#), [54](#), [59](#), [61–63](#), [68](#), [76](#)

BEFORE Versión previa al *merge*, equivalente a HEAD en Git. Representa el estado original del archivo antes de recibir los cambios. [iv](#), [38](#), [40](#), [45](#), [48–52](#), [54](#), [59](#), [61–63](#), [68](#), [76](#)

archivo binario Tipo de archivo que almacena datos en formato no legible para humanos, codificados en binario. En el desarrollo de videojuegos, se utiliza para representar recursos como texturas, modelos 3D, sonidos o escenas serializadas.. [5](#), [68](#), [76](#)

asset Cualquier archivo utilizado en un proyecto de desarrollo de un videojuego, incluyendo texturas, modelos 3D, scripts, materiales, escenas, etc. En el contexto de Unity todos se ubican dentro de `/Assets`. [8–11](#), [14](#), [21](#), [68](#)

componente Módulo funcional que se puede adjuntar a un GameObject en Unity3D para dotarlo de comportamientos específicos como renderizado, física, scripts, etc.. [3](#), [8](#), [34](#), [37](#), [38](#), [40–42](#), [45](#), [47](#), [50](#), [52](#), [57](#), [75](#), [76](#), [82](#)

conflicto Situación que se produce cuando dos o más desarrolladores modifican simultáneamente la misma parte de un archivo y el sistema de control de versiones no puede fusionar automáticamente los cambios.. [iv](#), [v](#), [1–3](#), [6](#), [28–42](#), [44–57](#), [59–71](#), [74–76](#), [82](#), [83](#)

editor Herramienta de desarrollo integrada en Unity3D que permite crear, modificar y gestionar todos los aspectos de un proyecto de videojuego.. [3](#)

escena Unidad principal de organización en Unity. Contiene un conjunto de objetos (GameObjects), luces, cámaras y otros componentes que conforman un entorno o nivel del proyecto.. [iv](#), [v](#), [28–34](#), [37](#), [38](#), [40–42](#), [44–55](#), [57](#), [59](#), [61](#), [62](#), [69–71](#), [74–77](#)

- GameObject** Entidad básica en Unity3D que representa un objeto en la escena y puede contener múltiples componentes que definen su comportamiento y propiedades.. [iv](#), [3](#), [8](#), [10](#), [21](#), [28](#), [30](#), [32–34](#), [37](#), [38](#), [40–42](#), [44](#), [45](#), [47–52](#), [54](#), [57](#), [59](#), [74–76](#), [82](#), [83](#)
- Git** Sistema de control de versiones distribuido creado por Linus Torvalds en 2005. [1–3](#), [5](#), [6](#), [10](#), [11](#), [13](#), [17](#), [21](#), [29](#), [30](#), [32](#), [33](#), [36–38](#), [40](#), [52–54](#), [57](#), [58](#), [60–62](#), [68](#), [71](#), [75](#), [76](#), [82](#), [83](#), [88](#)
- GUID** Identificador único global (*Globally Unique Identifier*) asignado por Unity a cada asset para mantener referencias internas coherentes, incluso tras cambios de nombre o ubicación. [68](#), [76](#)
- Juegos AAA** Término utilizado en la industria del videojuego para referirse a títulos desarrollados y publicados por grandes estudios con altos presupuestos, equipos numerosos y estándares de producción comparables a los de las superproducciones cinematográficas. Los juegos AAA suelen tener gráficos avanzados, una narrativa elaborada, campañas de marketing masivas y un enfoque comercial orientado a alcanzar amplias audiencias.. [5](#)
- material** Asset de Unity3D que define las propiedades visuales de una superficie, incluyendo color, textura, reflexión y otros efectos de renderizado.. [32](#)
- merge** Proceso de fusionar cambios de diferentes ramas o versiones de un proyecto en control de versiones.. [10](#)
- namespace** Contenedor lógico usado en programación para agrupar identificadores y evitar colisiones de nombres. Facilita la organización jerárquica y modular del código. [36](#)
- Perforce** Sistema de control de versiones centralizado utilizado en entornos profesionales, especialmente en desarrollo de videojuegos y grandes empresas. Ofrece alta eficiencia en gestión de archivos binarios y grandes repositorios. [5](#), [13](#)
- Prefab** Recurso reutilizable en Unity3D que permite crear múltiples instancias de un GameObject con propiedades y componentes predefinidos.. [2](#), [8](#), [14](#), [15](#), [21](#), [30](#), [32](#), [34](#), [41](#), [42](#), [52](#), [60](#), [61](#), [68](#)
- Scrum** Metodología ágil de gestión de proyectos centrada en la entrega incremental y colaborativa de productos. Organiza el trabajo en ciclos cortos llamados *sprints*, donde un equipo multidisciplinar colabora para alcanzar objetivos definidos. Incluye roles como *Product Owner*, *Scrum Master* y *Development Team*, así como eventos clave como reuniones diarias, revisiones de sprint y retrospectivas. [19](#)

SVN Subversion (SVN) es un sistema de control de versiones centralizado. Permite a varios usuarios gestionar cambios en proyectos compartidos, aunque con menos flexibilidad que sistemas distribuidos como Git. 5, 13

textura Imagen utilizada para aplicar detalles visuales a la superficie de objetos 3D en Unity3D.. 32

Unity Motor de desarrollo multiplataforma para crear videojuegos y aplicaciones interactivas en 2D y 3D. Ofrece herramientas integradas para gráficos, física, audio y scripting. iv, 1, 2, 4, 6–21, 23, 28–31, 33, 34, 36, 37, 40, 41, 44, 45, 47, 50, 51, 55, 56, 59, 63, 66–71, 75–78, 80, 83, 88

UX/UI Término que agrupa **UX** (User Experience o experiencia de usuario) y **UI** (User Interface o interfaz de usuario). UX se enfoca en cómo se siente y funciona una interacción, mientras que UI se centra en cómo luce y cómo se presenta visualmente. Ambos conceptos trabajan juntos para lograr productos intuitivos, usables y atractivos. 1

YAML Formato de serialización de datos legible, usado por Unity3D para guardar escenas, prefabs y assets. Sensible a indentación. Significa *YAML Ain't Markup Language*. 1, 2, 8, 10, 13, 16, 20, 21, 28, 29, 33, 34, 37, 38, 40, 52, 55, 57, 60, 61, 67, 68, 75–77, 82

Bibliografía

- [1] Electronic Arts, “Frostbite engine,” <https://www.ea.com/frostbite>, 2025, Último acceso: agosto 2025.
- [2] Rockstar Games, “Rockstar advanced game engine (rage),” <https://www.rockstargames.com/technology>, 2025, Último acceso: agosto 2025.
- [3] Epic Games, “Unreal engine 5,” <https://www.unrealengine.com/>, 2025, Último acceso: agosto 2025.
- [4] Crytek, “Cryengine,” <https://www.cryengine.com/>, 2025, Último acceso: agosto 2025.
- [5] Godot Engine Community, “Godot engine,” <https://godotengine.org/>, 2025, Último acceso: agosto 2025.
- [6] YoYo Games, “Gamemaker,” <https://gamemaker.io/>, 2025, Último acceso: agosto 2025.
- [7] *Unity3D: Motor de desarrollo de videojuegos multiplataforma*, Unity Technologies, 2024, Último acceso: agosto 2025. [En línea]. Disponible en: <https://unity.com/>
- [8] J. Gregory, *Game Engine Architecture*, 3rd ed. AKPeters/CRC Press, 2019.
- [9] Mastery Coding. (n.d.) Why unity is the best game engine for beginners. Accedido el 4 de agosto de 2025. [En línea]. Disponible en: <https://www.masterycoding.com/blog/unity-best-beginner-engine>
- [10] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Apress, 2014. [En línea]. Disponible en: <https://git-scm.com/book>
- [11] C. M. Pilato, B. Collins-Sussman, and B. W. Fitzpatrick, *Version Control with Subversion*, 2nd ed. Sebastopol, CA: O’Reilly Media, 2008.
- [12] The Git Project, “Git,” <https://git-scm.com/>, 2025, Último acceso: agosto 2025.

- [13] Perforce Software, “Helix core: Enterprise-class version control,” <https://www.perforce.com/products/helix-core>, 2025, Último acceso: agosto 2025.
- [14] Apache Software Foundation, “Apache subversion,” <https://subversion.apache.org/>, 2025, Último acceso: agosto 2025.
- [15] GitHub, “About git large file storage,” <https://docs.github.com/en/repositories/working-with-files/managing-large-files/about-git-large-file-storage>, 2025, Último acceso: agosto 2025.
- [16] Unknown, “Game development software engineering process life cycle,” *Journal of Software Engineering Research and Development*, November 2016.
- [17] Anonymous, “An extensive study on text serialization formats and methods,” *arXiv preprint arXiv:2505.13478v1*, 2009. [En línea]. Disponible en: <https://arxiv.org/html/2505.13478v1>
- [18] M. Cook, “Software engineering for automated game design,” *arXiv preprint arXiv:2004.01770*, 2020. [En línea]. Disponible en: <https://arxiv.org/abs/2004.01770>
- [19] Unity Technologies, “Unity 6,” <https://unity.com/releases/unity-6>, 2024–2025, Último acceso: agosto 2025.
- [20] Microsoft, “Visual studio 2022,” <https://visualstudio.microsoft.com/vs/>, 2022, Último acceso: agosto 2025.
- [21] GitHub, Inc., “Github,” <https://github.com/>, 2025, Último acceso: agosto 2025.
- [22] Instagantt, “Instagantt,” <https://instagantt.com/>, 2025, Último acceso: agosto 2025.
- [23] JGraph Ltd, “draw.io,” <https://app.diagrams.net/>, 2025, Último acceso: agosto 2025.
- [24] Mermaid JS Project, “Mermaid - markdown-like diagrams,” <https://mermaid-js.github.io/>, 2025, Último acceso: agosto 2025.
- [25] Unity Technologies, “Unity asset store,” <https://assetstore.unity.com/>, 2025, Último acceso: agosto 2025.
- [26] JustCreate / Unity Asset Store, “Low-poly simple nature pack,” <https://assetstore.unity.com/packages/3d/environments/landscapes/low-poly-simple-nature-pack-162153>, 2025, Último acceso: agosto 2025.
- [27] Scooter Software, “Beyond compare,” <https://www.scootersoftware.com/>, 2025, Último acceso: agosto 2025.

- [28] Joachim Eibl and KDE Project, “Kdiff3,” <https://kdiff3.sourceforge.net/>, 2025, Último acceso: agosto 2025.
- [29] Unity Technologies, “Unity smart merge (unityyamlmerge),” <https://docs.unity3d.com/6000.1/Documentation/Manual/SmartMerge.html>, 2025, Último acceso: agosto 2025.
- [30] —, “Unity version control,” <https://unity.com/products/unity-version-control>, 2025, Último acceso: agosto 2025.
- [31] Codice Software, “Plastic scm - distributed version control for big projects,” <https://www.plasticscm.com/>, 2020, Último acceso: agosto 2025.
- [32] —, “Codice software,” <https://www.codicesoftware.com/>, 2020, Último acceso: agosto 2025.
- [33] Unity Technologies, “Unity acquires codice software to bring version control to creators,” <https://blog.unity.com/news/unity-welcomes-codice-software-creator-of-plastic-scm>, 2020, Último acceso: agosto 2025.
- [34] F. G, “Gitmerge for unity,” <https://flashg.github.io/GitMerge-for-Unity/>, 2025, Último acceso: agosto 2025.
- [35] K. Schwaber and J. Sutherland, *The Scrum Guide*. Scrum.org, 2020. [En línea]. Disponible en: <https://scrumguides.org/>
- [36] Freelancermap, “Freelance rates in spain – insights for 2025,” <https://www.freelancermap.com/blog/self-employed-in-spain/>, 2025, accedido en julio de 2025.
- [37] SalaryExpert, “Software developer salary in spain,” <https://www.salaryexpert.com/salary/job/software-developer/spain>, 2025, accedido en julio de 2025.
- [38] Glassdoor, “Freelance developer salaries in madrid,” https://www.glassdoor.com.hk/Salaries/madrid-spain-freelance-developer-salary-SRCH_IL.0,12_IM1030_KO13,32.htm, 2025, accedido en julio de 2025.
- [39] SalaryExpert, “Project manager salary in spain,” <https://www.salaryexpert.com/salary/job/project-manager/spain>, 2025, accedido en julio de 2025.
- [40] Glassdoor, “Freelance project manager salaries in spain,” https://www.glassdoor.com/Salaries/spain-freelance-project-manager-salary-SRCH_IL.0,5_IN219_KO6,31.htm, 2025, accedido en julio de 2025.

- [41] G. . Graham, “Understanding the average day fee for a freelance senior project manager in europe,” <https://www.grant-graham.co.uk/en/grant-and-graham-blog/understanding-the-average-day-fee-for-a-freelance-senior-project-manager-in-europe>, 2025, accedido en julio de 2025.
- [42] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Patrones de diseño: elementos de software orientado a objetos reutilizable*. Addison-Wesley, 2002. [En línea]. Disponible en: https://biblioteca.unirioja.es/biba/mas_info.php?-titn=219053